



تقرير وظيفة برمجة تفرعية
اختصاص ذكاء فئة 7

High-Performance E-Commerce Backend Engine

تقدمة الطلاب :

- جودي سعيد همام اسماعيل
- ناي نضال المقداد
- أحمد فيصل غريب
- حيدر أحمد عيد
- حيان ياسين صالح

High-Performance E-Commerce Backend Engine

3	: (Project Introduction) مقدمة المشروع
3 ...	الطلب الأول: حماية البيانات المشتركة من التضارب (Concurrent Access & Data Integrity)
8	الطلب الثاني : إدارة الموارد الحاسوبية (Resource Management & Capacity Control)
11	 (Asynchronous Processing) الطلب الثالث: المعالجة غير المتزامنة
13	 (Batch Processing) الطلب الرابع : المعالجة الدفعية
18	Load Balancing & Auto Scaling الطلب الخامس :
25	(Distributed Caching) الطلب السادس: استراتيجية التخزين المؤقت
32	: (Distributed Concurrency Control) الطلب السابع : التحكم بالتزامن الموزع
40	:Transaction Integrity / ACID الطلب الثامن : سلامة المعاملات
42	 (Stress Testing Report) الطلب التاسع: اختبار الاستقرار تحت الضغط العالي
48	 (Bottlenecks) الطلب العاشر : الاختناقات

1. مقدمة المشروع (Project Introduction) :

في سوق التجارة الإلكترونية الحديث، تعد الكفاءة والسرعة والاعتمادية هي الركائز الأساسية لنجاح أي منصة. يهدف هذا المشروع إلى بناء **محرك خلفي عالي الأداء** قادر على معالجة آلاف الطلبات المتزامنة (Concurrent Requests) في نفس اللحظة دون انهيار النظام أو حدوث أخطاء في البيانات.

تم اختيار إطار العمل **Laravel (PHP)** لبناء هذا المحرك، وذلك بفضل بنيته التحتية المتطورة التي توفر أدوات قوية لإدارة الكاش، والعمليات المؤجلة (Queues)، والتعامل المتقدم مع قواعد البيانات. يركز المشروع بشكل أساسي على تحقيق **المتطلبات غير الوظيفية**

(Non-Functional Requirements - NFRs) مثل:

- **الموثوقية وسلامة البيانات (Data Integrity)** : ضمان عدم ضياع أو تضارب البيانات المالية والمخزون.
- **القابلية للتوسع (Scalability)** : قدرة النظام على التوسع مع زيادة عدد المستخدمين.
- **الزمن الاستجابي المنخفض (Low Latency)** : معالجة الطلبات بأسرع وقت ممكن.

الطلب الأول: حماية البيانات المشتركة من التضارب (Concurrent Access & Data Integrity)

المشكلة (The Race Condition Problem)

عندما يحاول أكثر من مستخدم شراء نفس المنتج في نفس الأجزاء من الثانية، يقرأ كلا الطلبين نفس قيمة المخزون الحالي من قاعدة البيانات مثلاً و يقوم كلا الطلبين بطرح الكمية المطلوبة وحفظ النتيجة، مما يؤدي إلى تحديث خاطئ للمخزون وهو ما يعرف برمجياً بـ **Race Condition** ويسبب كارثة البيع الزائد (Overselling) .

الحلول المقترحة (Proposed Solutions) :

الحل الأول: القفل المتفائل (Optimistic Locking) :

يعتمد على فكرة أن التضارب نادر الحدوث. يتم إضافة عميل **version** أو **timestamp** لجداول المنتجات. عند التحديث، يتم التأكد من أن النسخة لم تتغير. إذا تغيرت، يفشل الطلب ويُطلب من المستخدم إعادة المحاولة.

- **المميزات** : أداء سريع لأنه لا يغلق قاعدة البيانات.

- **العيوب:** سيئ جداً في حالات الضغط العالي والطلبات المتزامنة الكثيرة، لأن الكثير من الطلبات ستفشل وتطلب إعادة المحاولة.

الحل الثاني: القفل على مستوى قاعدة البيانات (Database-Level Locking (Pessimistic Locking :

يفترض هذا الحل أن التضارب سيحدث حتماً. عند قراءة المنتج، يتم حجز السطر في قاعدة البيانات (Row Lock) باستخدام lockForUpdate(). أي طلب آخر يحاول قراءة أو تعديل نفس السطر سيتوقف و ينتظر (Blocking) حتى ينتهي الطلب الأول. (Commit/Rollback).

- **المميزات:** يضمن سلامة البيانات بنسبة 100% على مستوى قاعدة البيانات مباشرة.
- **العيوب:** قد يسبب بطء إذا طالت فترة القفل، لأنه يستهلك اتصالات قاعدة البيانات. (DB Connections)

الحل المطبق :

في هذا المشروع سنعتمد على Pessimistic Locking داخل Database Transaction كحل أساسي ومباشر داخل Laravel لضمان عدم حدوث Overselling نهائياً، والسبب في ذلك:

1. **اتساق البيانات المطلق (ACID Compliance) :** يضمن أن عملية التحقق من المخزون والتعديل تتم ككتلة واحدة لا تتجزأ.
2. **منع الـ Overselling تماماً:** يضع المستخدم B في حالة انتظار (Blocking) تلقائي حتى ينتهي المستخدم A من إنقاص المخزون، ليقرأ B القيمة الجديدة الصحيحة .

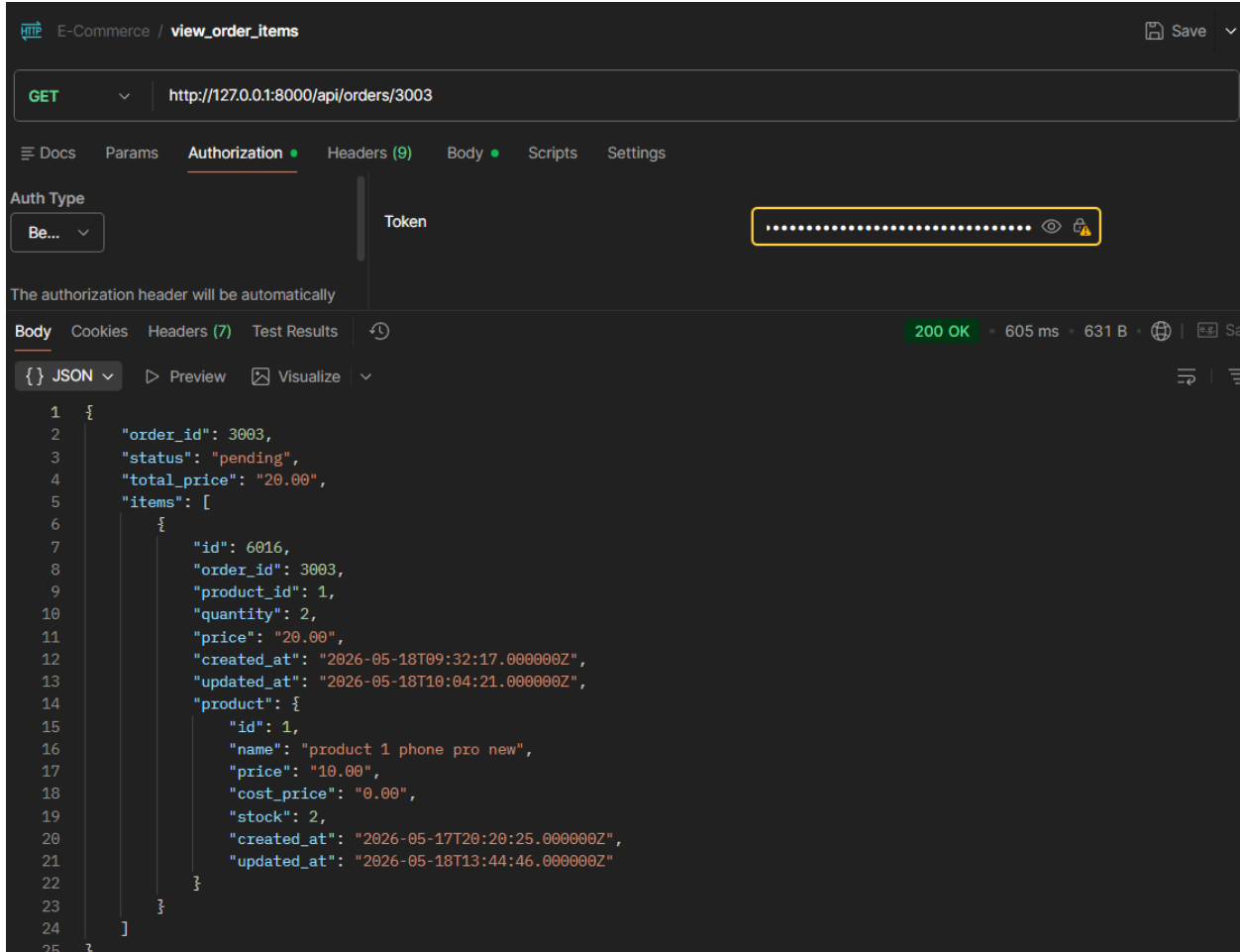
دليل اختبار التزامن والـ Race Condition (How to Test)

اختبار المشكلة :

لإثبات ثغرة الـ **Race Condition** في تابع (confirmOrderBroken)، استخدمنا Apache Server عبر بيئة Laragon لمعالجة الطلبات المتعددة بالتوازي (Multi-threading)، وأداة Apache JMeter لمحاكاة الهجوم الضغط المتزامن.

تم تجنب استخدام سيرفر PHP المدمج (artisan serve) لأنه أحادي المسار (Single-threaded) ويخفي مشكلة الـ Race Condition بتنفيذ الطلبات بالتوالي.

توضح هذه الصورة الطلب الذي نتعامل معه ، طلب لعنصرين من منتج واحد



المخزن للمنتج يحوي عنصرين فقط :

e_commerce.products: 20,053 rows total								
Next			Show all			Sorting		Columns (7/7)
id	name	price	cost_price	stock	created_at	updated_at		
1	product 1 phone pro new	10.0	0.0	2	2026-05-17 20:20:25	2026-05-18 10:09:22		

إعدادات إرسال طلب POST المتزامن لـ Endpoint الخاص بتأكيد الطلب، مع تفعيل الـ Synchronizing Timer لضمان خروج الطلبات معاً (كل الطلبات لنفس المنتج وبالكمية نفسها) :

نلاحظ ان جميع الطلبات تم قبولها :

وهكذا تم التأكد من حدوث الشجرة ، تم قبول جميع الطلبات رغم ان المخزن غير كافي :

e_commerce.products: 20,053 rows total								Next	Show all	Sorting	Columns (7/7)	Filter
id	name	price	cost_price	stock	created_at	updated_at						
1	product 1 phone pro new	10.0	0.0	0	2026-05-17 20:20:25	2026-05-18 13:44:46						

إختبار حل المشكلة :

إعدادات إرسال الطلب بالتابع (confirmOrderFixed):

HTTP Request

Name: HTTP Request

Comments:

Basic Advanced

Web Server

Protocol [http]: http Server Name or IP: 127.0.0.1 Port Number:

HTTP Request

POST Path: /E-Commerce/public/api/orders/3003/confirm-fixed Content encoding:

☐ Redirect Automatically ☒ Follow Redirects ☒ Use KeepAlive ☐ Use multipart/form-data ☐ Browser-compatible headers

Parameters Body Data Files Upload

Send Parameters With the Request:

Name:	Value	URL Encode?	Content-Type	Include Equals?

نلاحظ انه تم قبول طلب واحد فقط أي تم مراعاة نفاذ المخزون :

View Results Tree

Name: View Results Tree

Comments:

Write results to file / Read from file

Filename: Browse... Log/Display Only: ☐ Errors ☐ Successes Configure

Search: Case sensitive Regular exp. Search Reset

Text

Sampler result Request Response data

HTTP Request

Thread Name: Thread Group 1-4

Sample Start: 2026-05-18 17:55:18 EET

Load time: 876

Connect Time: 3

Latency: 868

Size in bytes: 568

Sent bytes: 578

Headers size in bytes: 342

Body size in bytes: 226

Sample Count: 1

Error Count: 0

Data type ("text"/"bin"/""): text

Response code: 200

Response message: OK

HTTPSampleResult fields:

ContentType: application/json

DataEncoding: null

الطلب الثاني : إدارة الموارد الحاسوبية (Resource Management & Capacity Control)

المشكلة التي يعالجها هذا الجزء هي كيفية إدارة الموارد الحاسوبية للنظام عند وجود عدد كبير من الطلبات المتزامنة، بحيث لا يستهلك التطبيق المعالج والذاكرة وقاعدة البيانات بشكل مفرط، ولا يتم تقليل الموارد بشكل يؤدي إلى بطء الاستجابة أو فشل الطلبات. في نظام التجارة الإلكترونية تظهر هذه المشكلة بشكل واضح في العمليات المتكررة مثل البحث عن المنتجات وإضافة منتج إلى السلة، لأن هذه العمليات قد تُنفَّذ من عدد كبير من المستخدمين بنفس الوقت.

في **Laravel** لا يوجد تعامل مباشر وواضح مع مفهوم **Thread Pool** لذلك تم الاعتماد على تقنيات بديلة لإدارة الموارد والتحكم بالضغط. استخدمنا فكرة قريبة من **Semaphore / bounded access** من خلال تحديد عدد الطلبات المسموح بها على العمليات الحساسة باستخدام **Rate Limiting**، بحيث لا يُسمح بعدد غير محدود من الطلبات بالدخول إلى نفس الجزء من النظام في نفس الوقت. كما تم تحسين طريقة تنفيذ العمليات نفسها، فتم نقل البحث من ذاكرة التطبيق إلى قاعدة البيانات باستخدام استعلامات محددة و **limit**، وتم تحسين عملية الإضافة إلى السلة باستخدام تحديث آمن للمخزون بشكل **atomic** لمنع مشاكل التزامن وتقليل العمل غير الضروري على قاعدة البيانات.

عمليات التحسين التي قمنا بها :

1- تحسين عملية البحث عن المنتجات :

في الكود غير المحسن كان البحث يجلب كل المنتجات من قاعدة البيانات إلى ذاكرة **Laravel** ثم يقوم بالفلتر داخل التطبيق. هذا الأسلوب يستهلك ذاكرة أكبر، ويزيد حجم البيانات، ويصبح بطيئاً عند وجود عدد كبير من المنتجات. أما في الكود المحسن، تم نقل عملية البحث إلى قاعدة البيانات نفسها حيث لا يتم تحميل جميع المنتجات إلى ذاكرة التطبيق، بل يتم تنفيذ شرط البحث داخل قاعدة البيانات باستخدام **limit** حتى لا يرجع النظام عدداً كبيراً من النتائج في كل طلب و تم تحديد الأعمدة المطلوبة فقط باستخدام **select** بدلاً من جلب كامل بيانات المنتج. بهذه الطريقة أصبح البحث أخف على الذاكرة والمعالج والشبكة.

2- تحسين عملية الإضافة إلى السلة :

في عملية الإضافة إلى السلة، المشكلة الأساسية هي أن عدة طلبات قد تصل بنفس الوقت وتحاول تعديل نفس المنتج أو نفس قيمة المخزون. لذلك تم استخدام أسلوب **atomic update** في الكود المحسن الفكرة التي تم التركيز هي تنفيذ التحقق من المخزون وإنقاصه ضمن عملية واحدة على مستوى قاعدة البيانات أي التحقق من المخزون والتعديل يحدثان كعملية واحدة، بدلاً من أن يقرأ التطبيق المخزون أولاً ثم يعدله لاحقاً. هذا يمنع حالات **race condition** ويقلل الضغط غير الضروري على قاعدة البيانات.

أيضاً تم استخدام **transaction** يضمن أن عملية الإضافة إلى السلة تتم كوحدة واحدة. فإذا حدث خطأ أثناء إضافة العنصر أو تحديث السعر، يتم التراجع عن العملية بدلاً من ترك البيانات بحالة غير صحيحة.

تم تحديد عدد ال **request** المتزامنة باستخدام **Rate Limiting** لمنع الضغط الزائد على النظام.

استخدمنا ثلاث أدوات أساسية للاختبار:

1- k6 : الهدف منه كان قياس أداء النظام قبل وبعد التحسين من ناحية زمن الاستجابة، نسبة فشل الطلبات، وعدد الطلبات التي يستطيع النظام معالجتها.

2- سكربت المراقبة monitor-v2.ps1 : هذا سكربت **PowerShell** استخدمناه لمراقبة استهلاك الموارد أثناء تشغيل اختبار **k6** يسجل مع الزمن استهلاك المعالج **CPU** والذاكرة **RAM** للعمليات المرتبطة بالمشروع **PHP** و **MySQL** و **Apache** الهدف منه هو الحصول على بيانات رقمية عن استهلاك الموارد أثناء الضغط

3- سكربت الرسم plot_resources.py : هذا سكربت Python استخدمناه لتحويل ملف القياسات الناتج من monitor-v2.ps1 إلى مخطط بياني.

الهدف منه هو عرض استهلاك CPU والذاكرة مع الزمن بشكل واضح، حتى نقدر نقارن بين حالة النظام قبل التحسين وبعده

الاختبار الذي قمنا به :

تم تنفيذ اختبار ضغط باستخدام k6 لمدة 60 ثانية وبمعدل 5 طلبات/ثانية، مع تخصيص 20 مستخدماً افتراضياً والسماح بالوصول حتى 80 مستخدماً عند الحاجة، بحيث يحاكي الاختبار حملاً مختلطاً يتكون من 70% طلبات بحث عن المنتجات و30% طلبات إضافة منتج إلى السلة.

نوع الاختبار constant-arrival-rate ، يعني k6 يحاول أن يحافظ على 5 عمليات بالثانية بغض النظر عن بطء الاستجابة.

تعليمات تشغيل اختبار كود ال Broken :

تشغيل مراقبة الموارد وتسجيل CPU/RAM بملف CSV

```
powershell -ExecutionPolicy Bypass -File .\tools\monitor-v2.ps1 -Out broken-metrics.csv -Seconds 70
```

تشغيل اختبار الضغط بـ k6

```
k6 run -e RATE=5 .\tests\k6\mixed-broken-v3.js
```

تحويل ملف الـ CSV إلى مخطط بياني للموارد

```
python .\tools\plot_resources.py broken-metrics.csv --title "Before resource management" --out broken-resources.png
```

تعليمات تشغيل اختبار كود ال Fixed :

```
powershell -ExecutionPolicy Bypass -File .\tools\monitor-v2.ps1 -Out fixed-metrics.csv -Seconds 70
```

```
k6 run -e RATE=5 .\tests\k6\mixed-fixed-v3.js
```

```
python .\tools\plot_resources.py fixed-metrics.csv --title "After resource management" --out fixed-resources.png
```

نتائج الاختبار :

قبل التحسين :

1- النظام لم يستطع المحافظة على معدل الطلبات المطلوب، وظهرت طلبات فاشلة بسبب timeout

2- نسبة فشل الطلبات وصلت إلى 25.80%

3- وظهر dropped_iterations = 84 ، وهذا يدل على أن النظام لم يستطع معالجة كل الطلبات المطلوبة ضمن الزمن المحدد.

بعد التحسين :

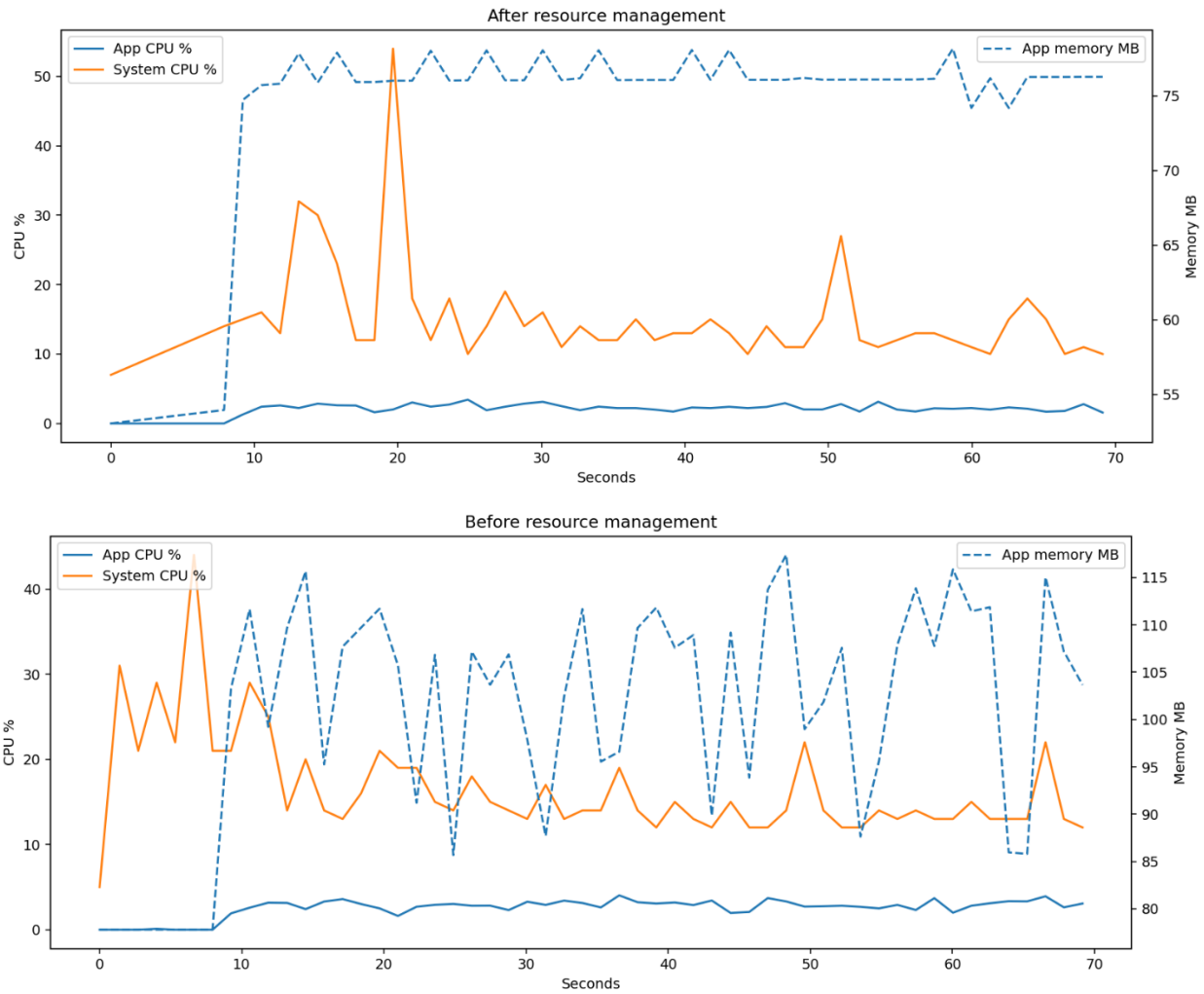
1- نسبة فشل الطلبات أصبحت 0%

2- وزمن الاستجابة p95 انخفض إلى 242ms

3- تم تنفيذ 300 طلب بنجاح

4- كما انخفض حجم البيانات المستقبلية إلى 420 KB فقط، ولم تظهر timeouts أو dropped iterations

ملاحظة حول استهلاك المعالج CPU وجدنا أن كل طلب HTTP في Laravel له كلفة ثابتة مثل تشغيل الـ middleware والـ routing والتعامل مع قاعدة البيانات وتجهيز الاستجابة. لذلك قد يظهر استهلاك CPU قريباً في الحالتين، لكن في الكود المحسن كان الـ cpu أكثر استقراراً .



جدول لمقارنة النتائج :

Metric	Before / Broken	After / Fixed
Failed requests	25.80%	0%
p95 response time	30s	242ms
Completed iterations	217	300
Dropped iterations	84	0
Received data	115 MB	420 KB
Request timeouts	Many timeouts	No timeouts
System behavior	Unstable under load	Stable under load

الطلب الثالث: المعالجة غير المتزامنة (Asynchronous Processing)

المشكلة التي يعالجها هذا الجزء

المشكلة الأساسية هي أن بعض العمليات التي تحدث بعد تأكيد الطلب مثل توليد الفاتورة وإرسال الإشعارات لا يحتاج المستخدم إلى انتظارها. في النظام غير المحسن، كانت هذه العمليات تنفذ داخل نفس دورة الطلب والاستجابة، مما يجعل المستخدم ينتظر +5 ثوانٍ قبل أن يتلقى أي رد. هذا يؤدي إلى بطء واضح في الاستجابة وضغط غير ضروري على الخادم تحت الحمل العالي.

عمليات التحسين التي قمنا بها

1 - النسخة (Broken) معالجة متزامنة وبلوكينج:

في الكود غير المحسن، كانت جميع العمليات تنفذ بشكل متسلسل داخل نفس الطلب:

- حفظ الطلب
- توليد الفاتورة (3 ثوانٍ)
- إرسال الإشعار (2 ثوانٍ)
- ثم فقط يُرسل الرد للمستخدم

المستخدم ينتظر كل هذه العمليات قبل أن يرى أي استجابة. إذا فشلت أي خطوة، يفشل الطلب كله. تحت الحمل العالي، كل PHP worker يبقى محجوزاً لمدة +5 ثوانٍ لكل طلب، مما يعني أن عدداً قليلاً جداً من الطلبات يمكن معالجتها بالتوازي.

2 - النسخة المحسنة (Fixed) معالجة غير متزامنة عبر Queue :

في الكود المحسن، تم تقسيم العمليات إلى:

المسار الحرج (Critical Path) ينفذ فوراً قبل إرسال الرد:

- تأكيد الطلب وتحديث الحالة إلى paid

المهام الخلفية (Background Tasks) تُرسل إلى Queue ولا ينتظرها المستخدم:

- توليد الفاتورة → ProcessInvoiceJob

- إرسال الإشعار → SendNotificationJob

يتلقى المستخدم الرد فوراً بعد تأكيد الطلب، بينما يعالج Workers هذه المهام في الخلفية بشكل مستقل.

3 - آلية إعادة المحاولة والـ Dead Letter Queue :

تم تكوين كل Job بـ 3 محاولات مع Exponential Backoff:

- المحاولة الأولى: انتظار 5 ثوانٍ

- المحاولة الثانية: انتظار 10 ثانية

- المحاولة الثالثة: انتظار 15 ثانية

بعد استنفاد جميع المحاولات، ينتقل الـ Job إلى جدول failed_jobs الذي يمثل الـ Dead Letter Queue، حيث يمكن مراجعته يدوياً أو إعادة تشغيله لاحقاً.

4 - عزل الـ Queues :

تم تخصيص queue منفصل لكل نوع من المهام:

- invoices queue → لمهام توليد الفواتير

- notifications queue → لمهام الإشعارات

هذا يعني أن ضغطاً على الإشعارات لن يؤخر توليد الفواتير والعكس صحيح. يتم تشغيل worker مستقل لكل queue .

أداة الاختبار المستخدمة

تم استخدام **Apache JMeter** لاختبار الأداء، حيث تم إرسال 50 طلباً متزامناً على كلا النسختين وقياس زمن الاستجابة والـ Throughput ونسبة الأخطاء.

نتائج الاختبار

قبل التحسين Synchronous Broken :

النظام يعالج كل طلب بشكل كامل داخل دورة الطلب، مما يجعل كل PHP worker محجوزاً لمدة +5 ثوانٍ. تحت حمل 50 مستخدماً متزامناً، تراكمت الطلبات وارتفع زمن الاستجابة بشكل كبير.

بعد التحسين Asynchronous Fixed :

النظام يُنهي المسار الحرج فوراً ويُرسل المهام الثقيلة إلى Queue. PHP worker يتحرر بسرعة لاستقبال الطلب التالي، مما يرفع ال-Throughput ويخفض زمن الاستجابة بشكل ملحوظ.

جدول مقارنة النتائج

Metric	Before / Broken	After / Fixed
Average response time	7,546 ms	3,076 ms
Min response time	5,858 ms	1,017 ms
Max response time	10,095 ms	5,247 ms
Error %	0.00%	2.80%
Throughput	3.5 req/sec	5.7 req/sec
Processing mode	Synchronous	Asynchronous
Background jobs	None	Invoice + Notification

الطلب الرابع : المعالجة الدفعية (Batch Processing)

في أنظمة التجارة الإلكترونية ذات الحجم الكبير، تُعدّ عملية إنشاء تقارير المبيعات اليومية من أكثر العمليات استهلاكاً للموارد أن كنا نريد معالجتها بالطرق التقليدية (أخذ السجلات كاملة دفعة واحدة ثم معالجتها) و ستواجهنا مشكلتين :

1 - Memory Exhaustion : يؤدي تحميل عشرات الآلاف من السجلات إلى الذاكرة العشوائية دفعة واحدة إلى استهلاك مفرط للموارد، مما قد يتسبب في انهيار السيرفر وتوقف الخدمة بالكامل.

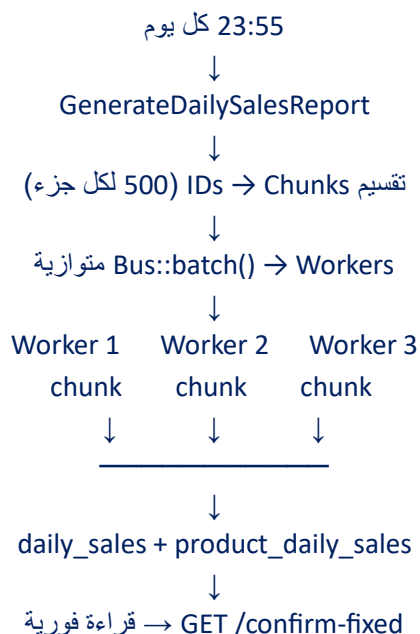
2 - Request Timeout : تفرض بيانات الويب حداً أقصى لزمن معالجة طلب HTTP عند تجاوز هذا الحد بسبب ضخامة البيانات، ينقطع الاتصال تلقائياً قبل إتمام العملية، مما يجعل التقرير غير قابل للاسترداد.

لمعالجة هاتين المشكلتين تم اعتماد Batch Processing .

بنية النظام :

يعتمد النظام على بنية مكونة من ثلاث طبقات تعمل بشكل متسلسل ومتواز:

- 1 - Chunk Generation :** يتم تشغيل GenerateDailySalesReport تلقائياً كل يوم عند الساعة 23:55 عبر Laravel Scheduler تقوم هذه Job بسحب بسحب جميع معرفات الطلبات الدفوعة لليوم و تقسيمها على دفعات بحجم 500 سجل لكل جزء إرسالها دفعة واحدة إلى Bus::batch() ثم تقسيمها إلى أجزاء متوازية Workers (مبدأياً علينا فتح تيرمنال لكل ووركر نريده).
- 2 - Processing :** ProcessSalesReportChunk تستلم كل جزء على حدة وتعالجه باستقلالية تامة تقوم بتحميل الطلبات مع عناصرها ومنتجاتها، إجراء الحسابات اللازمة (الأرباح، صافي الربح، عدد الطلبات ...) ثم حفظ النتائج في جدول daily_sales و product_daily_sales باستخدام نمط Upsert لضمان سلامة البيانات تحت التوازي.
- 3 - Query :** عند طلب التقرير عبر fixedReport النظام يقرأ فقط من الجداول الجاهزة ويعيد النتيجة فوراً لا يجري أي حساب و الجداول لن تكون كبيرة ستكون إحصائيات تتعلق بعدد المنتجات.



ماذا قمنا :

أولاً القرارات المتعلقة بالتوازي والبنية:

لماذا Queue وليس تنفيذ مباشر؟ Laravel لا تدعم تشغيل العمليات بشكل متوازي مباشرة داخل HTTP request ، لذلك تم اعتماد نظام ال Queue كوسيط يستلم ال Jobs ويوزعها على Workers تعمل باستقلالية تامة في الخلفية.

لماذا 3 طبقات وليس Job واحدة؟ الفصل بين طبقة التوليد (Generation) وطبقة المعالجة (Processing) كان قراراً مقصوداً. عملية إنشاء ال Workers وتسليمها ال Chunks تستغرق وقتاً و كون العملية خلفية تعمل في وقت محدد (23:55)، فإن فصل المسؤوليات يجعل كل طبقة مركزة على مهمة واحدة فقط، مما يسهل قراءة الكود و إصلاحه ويزيد الأداء.

لماذا (Bus::batch() وليس dispatch عادي) ؟ Bus::dispatch() يرسل Job واحدة في كل مرة يعني الـ Workers تعالج Chunks بشكل تسلسلي. أما Bus::batch() فيرسل جميع الـ Jobs دفعةً واحدة، مما يسمح لعدة Workers بمعالجة عدة Chunks في نفس الوقت وهذا هو جوهر التوازي في النظام. إضافةً إلى ذلك، يوفر batch تتبعاً تلقائياً لتقدم الـ Jobs وإدارة منظمة للأخطاء. أي كل ما ينتهي Worker سيستلم الـ chunk جديد .

ثانياً حجم الـ Chunk:

لم يكن اختيار حجم الـ Chunk عشوائياً بل تم اختياره للموازنة بين :

- عدد صغير جداً مثلاً 50 مما يُثقل النظام ويزيد عدد اتصالات قاعدة البيانات
- عدد كبير جداً مثلاً 5000 يعيدنا لمشكلة استنزاف الذاكرة

لذلك تم اختيار 500 كقيمة وسطى تُبقي استهلاك الذاكرة لكل الـ Jobs محدوداً مع إبقاء عدد الـ Worker ويجب الملاحظة بأن هذا الرقم يتفاوت من نظام لآخر حسب البيانات و الموارد التي يمتلكها كل نظام.

ثالثاً Upsert وليس Insert عادي؟

بما أن عدة Workers تعمل بالتوازي، فمن الممكن أن يكون نفس المنتج موجوداً في أكثر من Chunk في نفس الوقت. مثلاً:

- Worker 1 حسب: منتج 5 باع 300 وحدة
- Worker 2 حسب: منتج 5 باع 200 وحدة

لو استخدمنا insert أو update عادي، سيكتب أحد الـ Workers فوق الآخر والنتيجة ستكون 300 أو 200 بدلاً من 500 — أي بيانات خاطئة.

الحل كان استخدام upsert مع DB::raw على هذا الشكل:

```
'quantity_sold' => DB::raw('quantity_sold + VALUES(quantity_sold)')
```

هذا يعني: بدلاً من استبدال القيمة أضف عليها والعمليّة atomic على مستوى قاعدة البيانات أي لا يمكن لـ Worker آخر أن يتدخل في منتصفها النتيجة: $500 = 200 + 300$

وإذا كان السجل غير موجود أصلاً، يقوم بـ Insert تلقائياً. وإذا كان موجوداً، يقوم بـ Update تراكمي كل هذا في عملية واحدة آمنة.

رابعاً 23:55 وليس منتصف الليل؟

لو شغلنا الـ Job عند 00:00 أو 00:05، فإن today() في Laravel سيعطي اليوم الجديد يعني سنعالج طلبات يوم فارغ، وبيانات أمس تضيع للأبد. تشغيلها عند 23:55 يضمن أن today() لا يزال يشير لليوم الصحيح.

المقارنة بين brokenReport و fixedReport :

الجانب	brokenReport	fixedReport
آلية العمل	يحمل كل الطلبات في الذاكرة ويحسب مباشرة	يقرأ من جداول جاهزة مسبقاً
الذاكرة	استهلاك مفرط يتناسب مع عدد الطلبات	ثابت ومحدود بغض النظر عن حجم البيانات
زمن الاستجابة	يزداد مع ازدياد الطلبات	فوري دائماً
تحت الضغط	يتعطل	يعمل بثبات
Timeout	محتمل جداً	مستحيل
الحسابات	تتم لحظة الطلب	تمت مسبقاً في الخلفية

نتائج اختبار الأداء JMeter :

تم اختبار النظام باستخدام Apache JMeter على 3000 طلب موزع عليها 6000 عنصر مع 100 مستخدم متزامن، وكانت النتائج كالتالي:

metrics	brokenReport	fixedReport (processing)	fixedReport (query)
Average	42,788 ms	26,368 ms	17,569 ms
Min	1,056 ms	623 ms	529 ms
Max	82,903 ms	52,879 ms	34,224 ms
Throughput	1.1/sec	1.6/sec	2.3/sec
Error %	0%	0%	0%

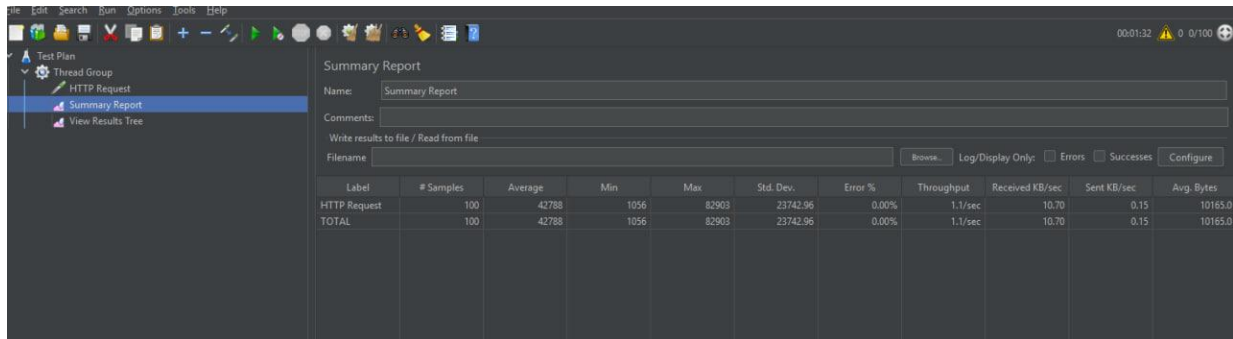
ملاحظة: لتشغيل ال broken يكفي <http://localhost:8000/api/sales/report/confirm-broken>

اما لتشغيل fixed فيوجد خطوتين generate مع فتح تيرمنالز و workers و هنا سيبدأ بتوليد و الحفظ داخل الداتا ثم confirm-fixed

<http://localhost:8000/api/sales/report/generate>

<http://localhost:8000/api/sales/report/confirm-fixed>

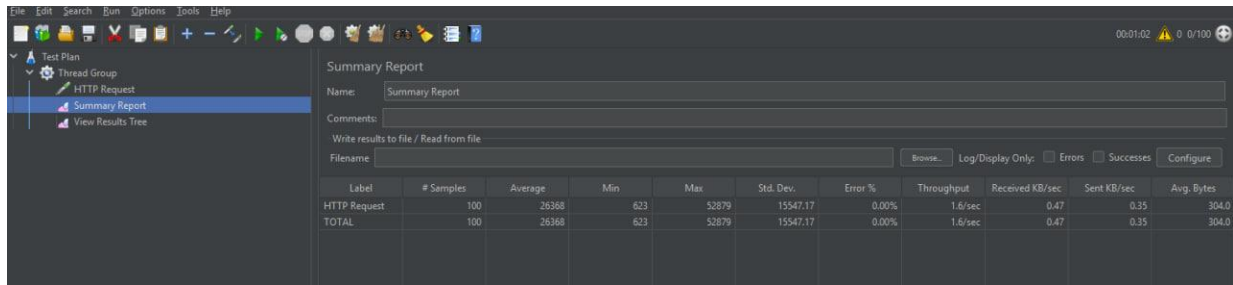
تحليل النتائج:



The screenshot shows the JMeter Summary Report for a test plan named 'brokenReport'. The report includes a table with the following data:

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	100	42788	1056	82903	23742.96	0.00%	1.1/sec	10.70	0.15	10165.0
TOTAL	100	42788	1056	82903	23742.96	0.00%	1.1/sec	10.70	0.15	10165.0

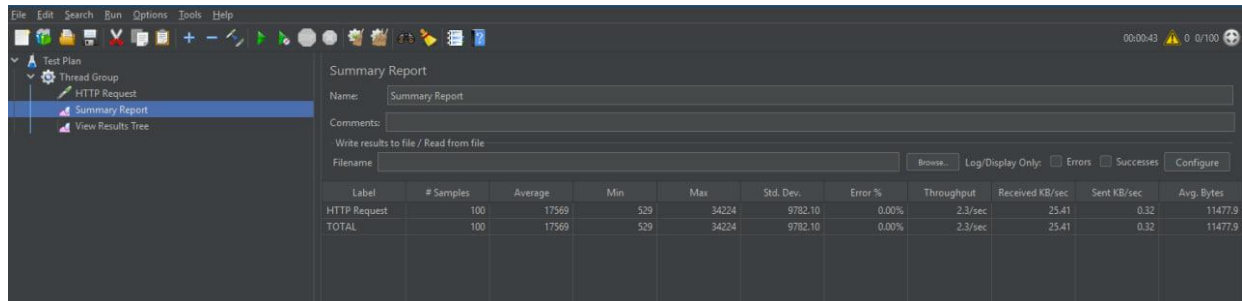
brokenReport (42,788ms average) الأبطأ لأنه يحتمل كل السجلات في الذاكرة ويجري الحسابات كاملة داخل الـ HTTP request مباشرة كل مستخدم يسبب ضغطاً كاملاً على السيرفر وقاعدة البيانات في نفس الوقت.



The screenshot shows the JMeter Summary Report for a test plan named 'fixedReport'. The report includes a table with the following data:

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	100	26368	623	52879	15547.17	0.00%	1.6/sec	0.47	0.35	304.0
TOTAL	100	26368	623	52879	15547.17	0.00%	1.6/sec	0.47	0.35	304.0

fixedReport (26,368ms average) أسرع من brokenReport لأن الحسابات تجري في الخلفية بشكل موزع، لكنه لا يزال يستغرق وقتاً لأن الـ Workers لا تزال تكتب في قاعدة البيانات.(عمل الخلفية).



The screenshot shows the JMeter Summary Report for a test plan named 'fixedReport'. The report includes a table with the following data:

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	100	17569	529	34224	9782.10	0.00%	2.3/sec	25.41	0.32	11477.9
TOTAL	100	17569	529	34224	9782.10	0.00%	2.3/sec	25.41	0.32	11477.9

fixedReport (17,569ms average) الأسرع لأن السيرفر لا يجري أي حساب يقرأ فقط من جداول جاهزة ويعيد النتيجة فوراً.

الفرق في الـ Throughput واضح: من 1.1/sec في brokenReport إلى 2.3/sec في fixedReport أي أن النظام الجديد يخدم ضعف عدد المستخدمين في نفس الوقت.

ملاحظة: تم الاختبار في بيئة محلية ذات موارد محدودة. في بيئة الإنتاج مع workers متعددة وقاعدة بيانات منفصلة، يكون الفارق في الأداء أكبر بكثير لصالح fixedReport

العمل على التوازي :

<pre> ueue. 2026-05-17 08:28:54 App\Jobs\GenerateDailyS h-performance-ecommerce/vendor RUNNING 2026-05-17 08:28:56 App\Jobs\GenerateDailyS alesReport 1s DONE 2026-05-17 08:28:56 App\Jobs\ProcessSalesRe portChunk RUNNING 2026-05-17 08:29:14 App\Jobs\ProcessSalesRe portChunk 17s DONE 2026-05-17 08:29:14 App\Jobs\ProcessSalesRe portChunk </pre>	<pre> ueue. 2026-05-17 08:28:57 App\Jobs\ProcessSalesRe portChunk RUNNING 2026-05-17 08:29:17 App\Jobs\ProcessSalesRe portChunk 20s DONE </pre>	<pre> ueue. 2026-05-17 08:28:57 App\Jobs\ProcessSalesRe portChunk RUNNING 2026-05-17 08:29:17 App\Jobs\ProcessSalesRe portChunk 20s DONE </pre>
---	---	---

تعليمة الجدولة :

```
Schedule::job(new GenerateDailySalesReport)->dailyAt('23:55');
```

الطلب الخامس : Load Balancing & Auto Scaling

1. نظرة عامة على المشكلة

في النظام الأساسي (قبل التحسين)، كانت جميع الطلبات تتجه إلى خادم واحد. هذا يعني نقطة فشل واحدة، واستهلاك غير محدود للموارد، وانتهاء تام تحت الضغط العالي.

لذا فنحن نهدف إلى:

- جعل التطبيق لا يعتمد على حالة محلية (Stateless) حتى يمكن توزيعه على خوادم متعددة
- توزيع الطلبات على عدة خوادم عبر Load Balancer كـ Nginx
- تقسيم اتصالات قاعدة البيانات إلى قراءة وكتابة
- بناء نظام مراقبة وتوسع تلقائي (Auto Scaling)
- إثبات التحسين عبر اختبارات أداء حقيقية باستخدام JMeter

2. جعل التطبيق عديم الحالة (Stateless)

إذا خزن الخادم بيانات الجلسة في ذاكرته المحلية، فإن الطلب التالي من نفس المستخدم يجب أن يصل إلى نفس الخادم وهذا يُلغي فائدة التوزيع. بنقل الجلسات إلى Redis (مخزن مشترك خارجي)، يصبح أي خادم قادراً على معالجة أي طلب.

تم تغيير قيمتين في ملف .env:

```
SESSION_DRIVER=redis
```

CACHE_STORE=redis

لماذا Redis تحديداً؟

- مخزن في الذاكرة (In-Memory) اي سرعة استجابة فائقة
- مشترك بين جميع الخوادم اي لا يوجد تبعية للخادم
- يستخدمه النظام أيضاً للـ Queue أي توحيد البنية التحتية

ملاحظة: نظام المصادقة مبني على JWT، وهو بطبيعته عديم الحالة لذا الخادم لا يحتاج لتخزين أي شيء للتحقق من الهوية، بل يتحقق من التوقيع الرقمي للـ Token.

3. التبديل من SQLite إلى MySQL

SQLite قاعدة بيانات مبنية على ملف واحد. في بيئة متعددة الخوادم، لا يمكن لأكثر من خادم الوصول لنفس الملف في وقت واحد بشكل موثوق. لذلك كان التبديل إلى MySQL (خادم قاعدة بيانات حقيقي).

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=ecommerce
DB_USERNAME=root
DB_PASSWORD=
```

4. تقسيم اتصالات قاعدة البيانات (Read/Write Split)

في التطبيقات الحقيقية، حوالي 80% من الاستعلامات هي قراءة (SELECT) لذا بتوجيه القراءة إلى خادم نسخة (Replica) والكتابة إلى الخادم الرئيسي (Primary) يتوزع الحمل ولا تصبح قاعدة البيانات عنق الزجاجة.

التعديل في config/database.php

```
'read' => ['host' => [env('DB_READ_HOST', '127.0.0.1')]],
'write' => ['host' => [env('DB_WRITE_HOST', '127.0.0.1')]],
'sticky' => true,
```

أهمية خيار sticky:

بدون sticky، بعد عملية كتابة، قد تذهب استعلامات القراءة إلى النسخة وتعود ببيانات قديمة (تأخر النسخ المتماثل). مع sticky، تظل القراءة على نفس خادم الكتابة طوال نفس الطلب.

في env. وضمن البيئة المحلية للتطوير

كلا المتغيرين DB_READ_HOST و DB_WRITE_HOST يشيران إلى 127.0.0.1 لمحاكاة النمط. في الإنتاج، يكفي تغيير DB_READ_HOST إلى عنوان IP الخادم النسخة.

5. إعداد Nginx كموازن للأحمال (Load Balancer)

Nginx هنا لا يقوم بتشغيل PHP بل يعمل ك Reverse Proxy يستقبل جميع الطلبات على المنفذ 8080 ويوزعها على خوادم Laravel المتعددة.

مسار الطلب:

المستخدم 8001 → Laravel:8001 → Nginx:8080 → أو 8002 أو 8003

خوارزمية التوزيع Least Connections :

تم اختيار least_conn بدلاً من round_robin لأن طلبات التجارة الإلكترونية تتباين كثيراً في وقت المعالجة (تصفح المنتجات سريع، بينما عملية الدفع والتحقق من المخزون أبطأ بكثير). Least Connections تضمن توجيه كل طلب جديد للخادم الأقل انشغالاً.

ملف الإعداد nginx.conf

```
worker_processes 1;

events { worker_connections 1024; }

http {
    error_log "C:/nginx/logs/error.log";
    access_log "C:/nginx/logs/access.log";

    upstream laravel_servers {
        least_conn;
        server 127.0.0.1:8001;
        server 127.0.0.1:8002;
        server 127.0.0.1:8003;
    }

    server {
        listen 8080;

        location /nginx_status {
            stub_status on;
            access_log off;
            allow 127.0.0.1;
            deny all;
        }
    }
}
```

```

}

location / {
    proxy_pass http://laravel_servers;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}
}
}

```

شرح كل جزء بالاعدادات

- `upstream laravel_servers`: يُعرّف مجموعة الخوادم الخلفية
- `least_conn`: خوارزمية التوزيع الخادم الأقل اتصالاً يستقبل الطلب التالي
- `listen 8080 Nginx`: يستمع على هذا المنفذ
- `proxy_pass`: يُحيل الطلب إلى مجموعة الخوادم
- `proxy_set_header`: يُمرر معلومات العميل الحقيقية للخادم الخلفي
- `location /nginx_status`: يكشف إحصائيات الاتصالات المباشرة يستخدمها Auto Scaler

الفحص الصحي السلبي (Passive Health Check)

Nginx يراقب صحة الخوادم بشكل سلبي تلقائياً: إذا فشل خادم في الاستجابة، يتوقف Nginx عن توجيه الطلبات إليه ويوزعها على الباقين. تم إثبات هذا عملياً بإيقاف خادمين من ثلاثة — النظام استمر في العمل بنسبة خطأ 0%.

7. نقطة الفحص الصحي /api/health/

```

Route::get('/health', function () {
    return response()->json([
        'status' => 'ok',
        'server' => gethostname(),
        'timestamp' => now(),
    ], 200);
});

```

الغرض

- تمكين Auto Scaler من التحقق من جاهزية الخادم الجديد قبل توجيه الطلبات إليه
- يُظهر اسم الخادم لتأكيد أن Nginx يوزع الطلبات فعلاً بين الخوادم المختلفة
- يُستخدم لإثبات الفحص الصحي السلبي بقتل خادم ومراقبة استمرار الاستجابة

8. نظام التوسع التلقائي (Auto Scaler)

Auto Scaler هو سكربت PHP يعمل في الخلفية ويراقب الحمل على الخوادم كل ثانية، ويتخذ قرار التوسع أو التقليل كل 5 ثوانٍ بحيث يقرأ Auto Scaler إحصائيات الاتصالات المباشرة من Nginx عبر:

http://127.0.0.1:8080/nginx_status

يُحلل الاستجابة باستخراج قيمة Active connections ثم يقسمها على عدد الخوادم للحصول على متوسط الحمل لكل خادم.

منطق قرار التوسع

- إذا كان متوسط الاتصالات لكل خادم < 25 → تشغيل خادم جديد (Scale Up)
- إذا كان متوسط الاتصالات لكل خادم ≥ 10 → إيقاف خادم زائد (Scale Down)
- خلاف ذلك → لا إجراء، عرض الحالة فقط

آلية التوسع

عند Scale Up:

- تشغيل خادم Laravel جديد على منفذ جديد باستخدام proc_open
- إضافة المنفذ الجديد إلى nginx.conf
- إعادة تحميل Nginx بأمر: `nginx.exe -p C:\nginx -s reload`

عند Scale Down:

- إيجاد PID العملية على المنفذ عبر netstat
- إنهاء العملية باستخدام taskkill
- حذف المنفذ من nginx.conf وإعادة تحميل Nginx

9. المراحل الثلاث للاختبار

المرحلة الأولى: بدون Auto Scaler (3 خوادم ثابتة، Nginx معطل التحديث)

في هذه المرحلة، Auto Scaler كان يُشغّل خوادم جديدة لكن Nginx لم يكن يوجه إليها الطلبات بسبب خطأ أمر الـ reload. الخوادم الأصلية الثلاثة كانت تتحمل العبء وحدها.

المقياس	التشغيل الأول	التشغيل الثاني	التشغيل الثالث
عدد العينات	2000	—	—
نسبة الأخطاء	42.25%	—	—
متوسط وقت الاستجابة	17,866ms	—	—
أقل وقت استجابة	2ms	—	—
أعلى وقت استجابة	37,787ms	—	—
الإنتاجية	20.9 طلب/ثانية	—	—

المرحلة الثانية: Auto Scaler مع حد أقصى 3 خوادم ديناميكية (6 إجمالاً)

بعد إصلاح أمر nginx reload، أصبح Auto Scaler يُشغل خوادم جديدة ويوجه الطلبات إليها فعلاً. الحد الأقصى كان 6 خوادم (3 أساسية + 3 ديناميكية).

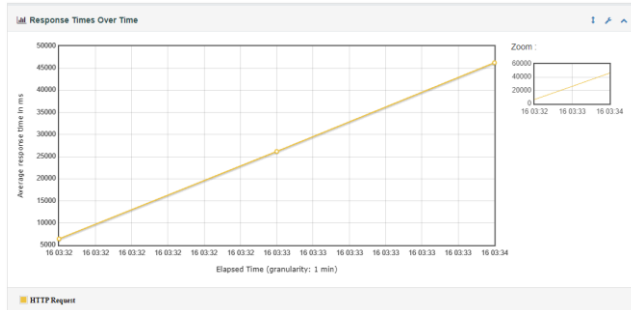
المقياس	التشغيل الأول	التشغيل الثاني	التشغيل الثالث
عدد العينات	—	2000	—
نسبة الأخطاء	—	36.75%	—
متوسط وقت الاستجابة	—	9,366ms	—
أقل وقت استجابة	—	2ms	—
أعلى وقت استجابة	—	23,741ms	—
الإنتاجية	—	27.1 طلب/ثانية	—

المرحلة الثالثة: Auto Scaler بخوادم لانهاية (range 8004–8099)

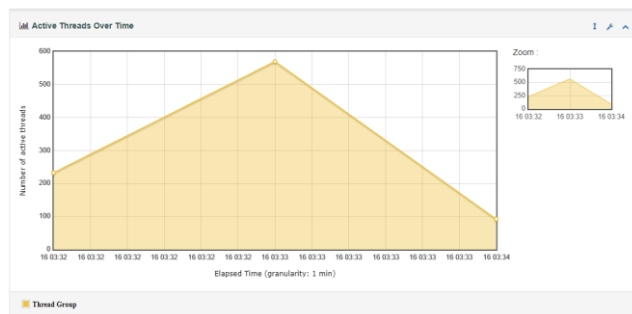
بتوسيع نطاق المنافذ الديناميكية من 3 إلى 96 خادماً ممكناً (range(8004, 8099)، أصبح النظام يُضيف خوادم بلا حد أقصى عملي حتى يتعامل مع الحمل الكامل.

المقياس	التشغيل الأول	التشغيل الثاني	التشغيل الثالث
عدد العينات	—	—	2000
نسبة الأخطاء	—	—	0.00% ✓
متوسط وقت الاستجابة	—	—	21,434ms
أقل وقت استجابة	—	—	271ms
أعلى وقت استجابة	—	—	48,569ms
الإنتاجية	—	—	18.9 طلب/ثانية

ملاحظة: ارتفاع متوسط وقت الاستجابة في المرحلة الثالثة مقارنة بالثانية طبيعي لأن النظام الآن يُعالج 100% من الطلبات بدلاً من رفض بعضها بسرعة. الاستجابة البطيئة الناجحة أفضل من الاستجابة السريعة الفاشلة.



نلاحظ ازدياد مدة معالجة الطلب مع مرور الزمن وذلك لأن عملية اعداد خادم جديد تتطلب وقتاً (لكن بفضل ذلك أصبحت جميع الطلبات ناجحة حتى مع بعض التأخير)



نلاحظ وصول عدد نياشب المعالجة لقمة في منتصف وقت المعالجة (حينها كان عدد الطلبات اكبر ما يمكن) وبعد معالجة الضغط تم إيقاف بعض الخوادم وبالتالي قل عدد النياشب العاملة

9. مقارنة ملخصة

بعد التحسين	قبل التحسين	الجانب
Redis مشترك	ملف محلي / قاعدة بيانات	الجلسات
MySQL (خادم)	SQLite (ملف)	قاعدة البيانات
3 + Nginx خوادم	خادم واحد	توزيع الحمل
قراءة / كتابة منفصلان	اتصال واحد	اتصالات DB
Auto Scaler تلقائي	يدوي	التوسع
%0.00	%42.25	نسبة الأخطاء تحت الضغط

الطلب السادس: استراتيجية التخزين المؤقت (Distributed Caching)

المشكلة:

تعتمد منصات التجارة الإلكترونية الحديثة على معالجة تدفقات ضخمة من الطلبات المتزامنة (High Concurrent Traffic) في التوابع مثل جلب المنتجات الأكثر مبيعاً (topSellingProducts) وعرض تفاصيل المنتج (cachedProduct)، والبحث عن منتج (searchCache) يكمن العائق الأساسي في إجهاد طبقة البيانات (Database Overload) نتيجة لكثرة الاستعلامات المتكررة والمعقدة.

هذه الاستعلامات تحوي على عمليات كثيرة أو عمليات تصبح معقدة عند تعرض النظام لضغط متزامن من مئات المستخدمين في نفس الثانية، تضطر قاعدة البيانات (MySQL) لإعادة حساب هذه العمليات بشكل متكرر لكل مستخدم، مما يؤدي إلى:

1. استهلاك كامل لطاقة المعالج (CPU Throttling) بداخل سيرفر قاعدة البيانات.
2. تراكم الطلبات في طوابير الانتظار وجعل زمن الاستجابة (Average Response Time) يرتفع بشكل كارثي.
3. حدوث أخطاء ونفاذ وقت الاتصال (Timeout Errors) واختناق منافذ الاتصال المتزامنة (Socket Exhaustion).

الحل الهندسي المقترح :

بناء طبقة تخزين مؤقت عازلة وموزعة (Distributed Caching Layer) متمثلة في سيرفر Redis المعتمد على الذاكرة العشوائية (RAM)، وبذلك بدلاً من معالجة الاستعلام المكلف مع كل طلب، يتم تنفيذه لمرة واحدة فقط (Warm-up)، ثم

تجميد وتخزين مصفوفة البيانات الجاهزة داخل الذاكرة العشوائية بمفتاح فريد (Cache Key) تصبح قراءة الطلبات التالية مباشرة من الـ RAM بزمان استجابة يقارب الصفر، دون إشغال قاعدة البيانات التقليدية

• ميزات الحماية البرمجية المضافة في الكود (Resilience & Monitoring) :

1. آلية الـ **Fallback الآمن** : تم تغليف عمليات الاتصال بـ Redis داخل كتل try/catch والتقاط الاستثناء Throwable. في حال سقوط سيرفر Redis أو فقدان الاتصال به، يتحول التطبيق تلقائياً وبسلاسة إلى قاعدة البيانات (database_fallback) دون أن يتوقف الموقع عن العمل أمام المستخدم النهائي (يتم تسجيل النتائج بال log لتوضيح إذا كنا نعتمد على الكاش أو الداتابيز في ارجاع الاستجابة).
2. مؤشرات كفاءة الذاكرة المؤقتة: (Cache Efficiency Metrics) لضمان دقة المعمارية المتبعة، تم دمج عدادات داخل نظام الكاش لمراقبة معدل الإصابة والإخفاق (Cache Hit/Miss Ratio)
3. إدارة سلامة واتساق البيانات (Data Consistency & Observers)
من مشاكل الكاش هي مشكلة "البيانات القديمة أو الميتة" (Stale Data) "لحل هذه المعضلة الهندسية وجعل الكاش ذكياً ومتسقاً تماماً مع قاعدة البيانات الحية، قمنا ببناء **ProductObserver** ليراقب الأحداث الحية للمنتجات ويقوم بعملية إبطال وتنظيف تلقائي للكاش فور حدوث أي تعديل على البيانات في الداتا بيز (من قبل الأدمن مثلاً).
4. استراتيجية صلاحية البيانات المؤقتة (Time-To-Live - TTL Strategy)
تم تحديد زمن بقاء البيانات في الذاكرة المؤقتة بشكل يضمن توازن بين سرعة استجابة النظام واتساق وصحة البيانات المعروضة للمستخدم .
 - المنتجات الأكثر مبيعاً (topSellingProducts) : تم ضبط مدة الـ TTL: عشر دقائق - Minutes10 ، لأن حركية مبيعات المتجر متغيرة باستمرار ، وتحديث القائمة كل 10 دقائق يضمن تزويد الزبائن بإحصائيات حية وشبه فورية لأحدث المنتجات الرائجة.
 - تفاصيل المنتج الفردي (cachedProduct) : تم ضبط مدة الـ TTL: ساعة كاملة - Minutes60 ، لأن صفحة تفاصيل المنتج (الاسم، الوصف، السعر المبدئي) تُعد من أكثر الصفحات زيارة من قبل الزبائن (Read-Heavy Action)، وفي الوقت نفسه هي بيانات شبه ثابتة (Static Data) ولا تتغير خلال الدقيقة أو الساعتين إلا نادراً بقرار من مدير المتجر.
 - البحث النصي المتعدد (searchCache) : تم ضبط مدة الـ TTL: عشر دقائق - Minutes10 ، بما أن احتمالات كلمات البحث التي يكتبها المستخدمون في المتجر غير محدودة، فإن توليد مفاتيح كاش لمدد طويلة (كساعات أو أيام) لكل كلمة بحث جديدة قد يسبب تضخماً واستهلاكاً هائلاً لذاكرة الـ Redis بدون حاجة فعلية. تحديد الـ TTL بـ 10 دقائق يضمن تدمير الاستعلامات القديمة تلقائياً وتفريغ الذاكرة باستمرار للمستخدمين النشطين حالياً.

منهجية اختبار التحمل والأداء (Load Testing Methodology)

تم استخدام أداة الفحص القياسية Apache JMeter لعمل محاكاة ومراقبة هجوم متزامن يحاكي السلوك البشري الحقيقي على السيرفر المحلي بدقة.

5. محددات وثوابت بيئة الاختبار: (Thread Group Configuration)

1. عدد المستخدمين الوهميين المتزامنين (Virtual Users / Threads): 100 مستخدم.
2. فترة الصعود وضخ الطلبات (Ramp-Up Period): 10 ثوانٍ.
3. عدد دورات تكرار الطلب (Loop Count): 3 دورات لكل مستخدم.
4. إجمالي عينات الفحص المستهدفة (Total Samples): 300 عينة ممررة بالتوازي لكل مسار على حدة لضمان العدالة الإحصائية.

استخدام الكاش في (topSellingProducts) :

عندما نقوم بعمل اختبار لنسخة التابع الذي لا يحوي على كاش بل يصل لقاعدة البيانات مباشرة ،

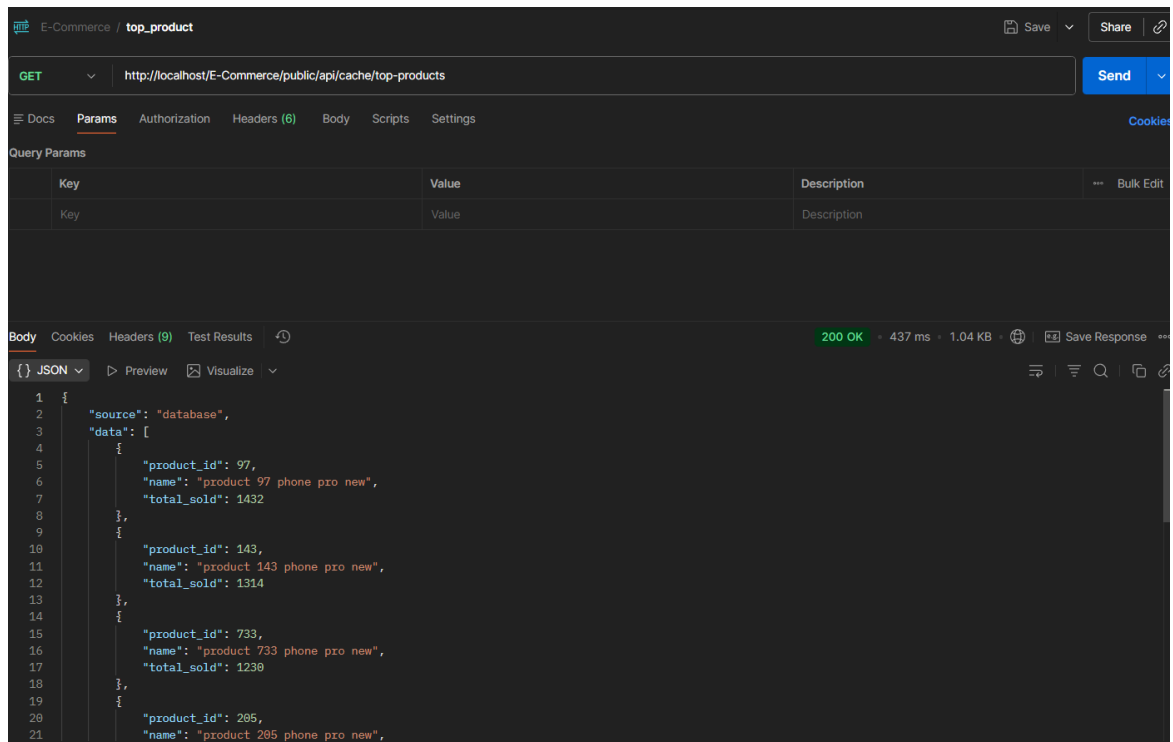
<http://localhost/E-Commerce/public/api/cache/top-products/broken>

نلاحظ النتائج التالية :

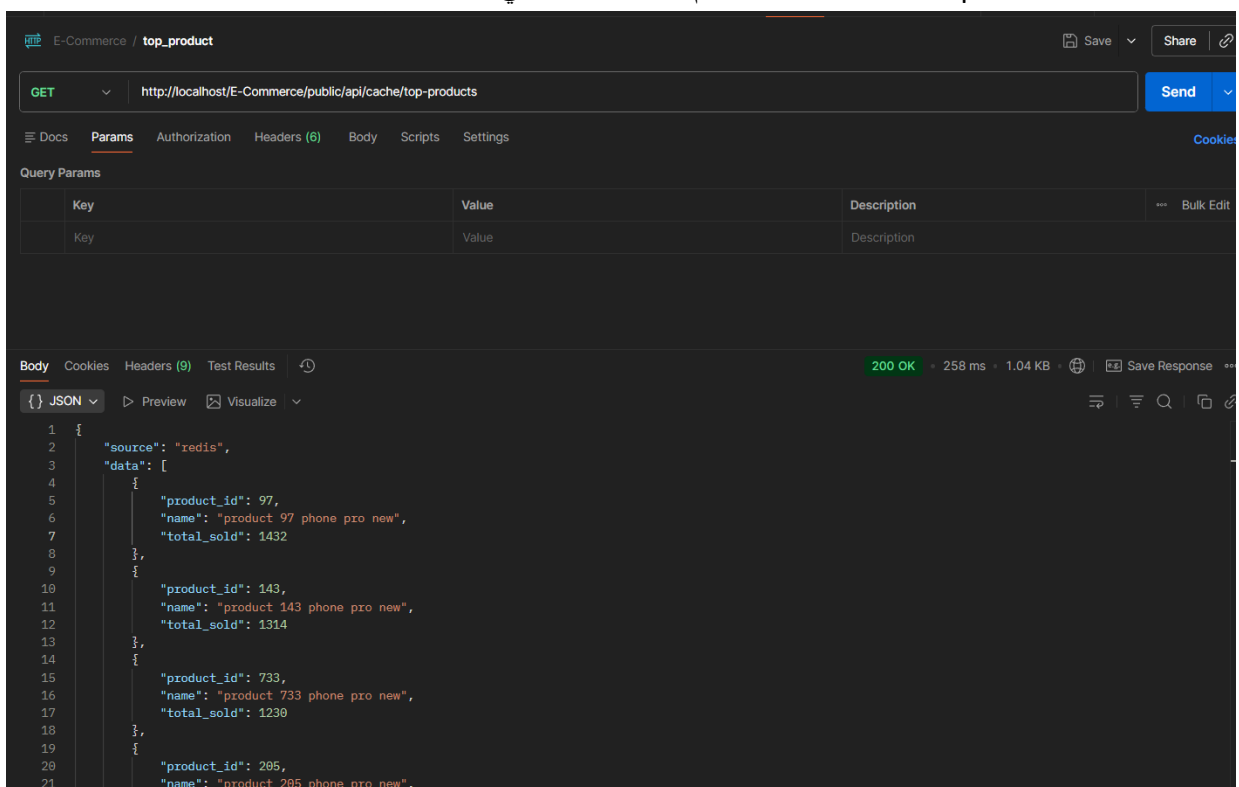
Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received ...	Sent KB/sec	Avg. Bytes
TTP Req...	300	1075	342	2782	492.68	0.00%	22.6/sec	23.84	3.52	1078.3
OTAL	300	1075	342	2782	492.68	0.00%	22.6/sec	23.84	3.52	1078.3

- معدل متوسط الاستجابة (Average) : أكثر من ثانية ليتم الرد على السيرفر
- معدل الانتاجية (Throughput) : السيرفر كان قادر على معالجة 22.6 طلب في الثانية فقط
- وصل في بعض الطلبات إلى 2782ملي ثانية (قريب من 3 ثوان)

والآن عندما نريد اختبار نسخة التابع الذي استخدمنا فيه كاش ، بداية سيكون الكاش فارغ عند أول request للمنتجات الأكثر مبيعاً، فيتم تحضير البيانات وجلبهم من ال DataBase لأول مرة فقط :



وعند طلب نفس ال request مرة أخرى يكون قد تم تخزين البيانات في redis :



الان عندما نقوم بعمل اختبار بعد تهيئة ال redis بالبيانات :

<http://localhost/E-Commerce/public/api/cache/top-products>

نلاحظ النتائج التالية :

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/...	Sent KB/sec	Avg. Bytes
HTTP Request	300	292	225	481	35.48	0.00%	28.1/sec	29.50	4.17	1075.4
TOTAL	300	292	225	481	35.48	0.00%	28.1/sec	29.50	4.17	1075.4

- متوسط وقت الاستجابة (Average) : تحسن ليصل إلى 292 مللي ثانية فقط

- معدل الإنتاجية (Throughput) : ارتفع ليصبح 28.1 طلب في الثانية.

- أقصى وقت استجابة (Max) : انخفض ليصبح 481 مللي ثانية كأعلى حد

في الصورة التالية نرى نسبة نجاح الكاش :

The screenshot shows a web browser interface with the URL `http://localhost/E-Commerce/public/api/cache/stats` and the method `GET`. The response body is displayed in JSON format:

```
{  "hits": 272,  "misses": 1,  "hit_ratio": "99.63%"}
```

توضيح ال log :

- قبل استخدام الكاش وبعد استخدام :

```
[2026-06-15 19:19:37] production.INFO: [Benchmark] بدون كاش DB طلب مباشر من جلب
[2026-06-15 19:19:37] local.INFO: [Benchmark] بدون كاش DB طلب مباشر من جلب
[2026-06-15 19:19:37] local.INFO: [Benchmark] بدون كاش DB طلب مباشر من جلب
[2026-06-15 19:19:37] local.INFO: [Benchmark] بدون كاش DB طلب مباشر من جلب
[2026-06-15 19:19:37] local.INFO: [Benchmark] بدون كاش DB طلب مباشر من جلب
[2026-06-15 19:19:37] local.INFO: [Benchmark] بدون كاش DB طلب مباشر من جلب
[2026-06-15 19:19:38] local.INFO: [Benchmark] بدون كاش DB طلب مباشر من جلب
[2026-06-15 19:19:38] local.INFO: [Benchmark] بدون كاش DB طلب مباشر من جلب
[2026-06-15 19:19:38] local.INFO: [Benchmark] بدون كاش DB طلب مباشر من جلب
[2026-06-15 19:19:38] local.INFO: [Benchmark] بدون كاش DB طلب مباشر من جلب
[2026-06-15 19:19:38] local.INFO: [Benchmark] بدون كاش DB طلب مباشر من جلب
[2026-06-15 19:19:38] local.INFO: [Benchmark] بدون كاش DB طلب مباشر من جلب
[2026-06-15 19:19:38] local.INFO: [Benchmark] بدون كاش DB طلب مباشر من جلب
[2026-06-15 19:19:38] local.INFO: [Benchmark] بدون كاش DB طلب مباشر من جلب
[2026-06-15 19:19:38] local.INFO: [Benchmark] بدون كاش DB طلب مباشر من جلب
[2026-06-15 19:25:49] local.INFO: [Benchmark] Redis جلب كاش من ال
[2026-06-15 19:25:49] local.INFO: [Benchmark] Redis جلب كاش من ال
[2026-06-15 19:25:49] local.INFO: [Benchmark] Redis جلب كاش من ال
[2026-06-15 19:25:49] local.INFO: [Benchmark] Redis جلب كاش من ال
[2026-06-15 19:25:49] local.INFO: [Benchmark] Redis جلب كاش من ال
[2026-06-15 19:25:49] local.INFO: [Benchmark] Redis جلب كاش من ال
[2026-06-15 19:25:49] local.INFO: [Benchmark] Redis جلب كاش من ال
[2026-06-15 19:25:49] local.INFO: [Benchmark] Redis جلب كاش من ال
[2026-06-15 19:25:49] local.INFO: [Benchmark] Redis جلب كاش من ال
[2026-06-15 19:25:49] local.INFO: [Benchmark] Redis جلب كاش من ال
[2026-06-15 19:25:49] local.INFO: [Benchmark] Redis جلب كاش من ال
[2026-06-15 19:25:49] local.INFO: [Benchmark] Redis جلب كاش من ال
[2026-06-15 19:25:49] local.INFO: [Benchmark] Redis جلب كاش من ال
[2026-06-15 19:25:49] local.INFO: [Benchmark] Redis جلب كاش من ال
```

استخدام الكاش في (cachedProduct) :

في السيناريو التقليدي عندما يضغط المستخدم على منتج معين لرؤية تفاصيله (النسخة بدون استخدام كاش) نقوم بعمل استعلام مباشر على قاعدة البيانات Product::find(\$product_id) في كل طلب. إذا قام 300 أو 500 مستخدم بفتح صفحة نفس المنتج في نفس الميكروثانية، ستجبر قاعدة البيانات على فتح مئات الاتصالات المتزامنة وقراءة القرص الصلب لنفس السجل مراراً وتكراراً، مما يرفع ال Response Time ويستهلك ال CPU تماماً .

<http://localhost/E-Commerce/public/api/cache/product/broken/1>

Write results to file / Read from file											
Filename						Browse...		Log/Display Only:		☐ Errors	☐ Successes
Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes	
HTTP Request	300	382	235	540	42.74	0.00%	27.8/sec	14.79	4.24	544.3	
TOTAL	300	382	235	540	42.74	0.00%	27.8/sec	14.79	4.24	544.3	

أما باستخدام الكاش عند تكرار نفس الفحص ب 300 طلب ، تمت خدمة اول طلب من الداتا بيز (لتهيئة ال redis) وبقية الطلبات من ال redis بسرعة أكبر :

Write results to file / Read from file											
Filename						Browse...		Log/Display Only:		☐ Errors	☐ Successes
Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes	
HTTP Request	300	292	225	481	35.48	0.00%	28.1/sec	29.50	4.17	1075.4	
TOTAL	300	292	225	481	35.48	0.00%	28.1/sec	29.50	4.17	1075.4	

استخدام الكاش في (searchCache) :

قبل استخدام الكاش ، عندما يطلب مستخدم كلمة معينة، السيرفر يقوم بعمل get() لكل المنتجات ويضعهم في ذاكرة السيرفر (PHP RAM)، ثم يبدأ بفلترتهم واحداً تلو الآخر باستخدام دالة ال filter().

<http://localhost/E-Commerce/public/api/products-search/broken>

و parameters التالية :

q : product , limit :20

تحت ضغط العديد من المستخدمين :

Write results to file / Read from file											
Filename						Browse...		Log/Display Only:		☐ Errors	☐ Successes
Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes	
HTTP Request	300	1712	565	3861	713.24	0.00%	20.3/sec	45.85	3.32	2317.4	
TOTAL	300	1712	565	3861	713.24	0.00%	20.3/sec	45.85	3.32	2317.4	

عند استخدام الكاش ، المستخدم سيبحث عن الكلمة أول مرة وسيكون الريدس فارغ ، سيتم احضار المنتجات من قاعدة البيانات وتخزينها في ال Redis لمدة وضعها ك 10 دقائق ، بقية الطلبات الذين يبحثون عن نفس الكلمة سيتم احضارها من ال Redis :

<http://localhost/E-Commerce/public/api/products-search>

و parameters التالية :

q : product , limit :20

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received ...	Sent KB/sec	Avg. Bytes
TTP Req...	300	1075	342	2782	492.68	0.00%	22.6/sec	23.84	3.52	1078.3
OTAL	300	1075	342	2782	492.68	0.00%	22.6/sec	23.84	3.52	1078.3

النتائج بال log عند استخدام الكاش :

```

175 local.INFO: [Benchmark] الـ Redis كلمة: 'product'
176 production.INFO: [Benchmark] الـ Redis كلمة: 'product'
177 local.INFO: [Benchmark] الـ Redis كلمة: 'product'
178 local.INFO: [Benchmark] الـ Redis كلمة: 'product'
179 local.INFO: [Benchmark] الـ Redis كلمة: 'product'
180 local.INFO: [Benchmark] الـ Redis كلمة: 'product'
181 local.INFO: [Benchmark] الـ Redis كلمة: 'product'
182 local.INFO: [Benchmark] الـ Redis كلمة: 'product'
183 local.INFO: [Benchmark] الـ Redis كلمة: 'product'
184 local.INFO: [Benchmark] الـ Redis كلمة: 'product'
185 local.INFO: [Benchmark] الـ Redis كلمة: 'product'
186 local.INFO: [Benchmark] الـ Redis كلمة: 'product'
187 local.INFO: [Benchmark] الـ Redis كلمة: 'product'
188 local.INFO: [Benchmark] الـ Redis كلمة: 'product'
189 local.INFO: [Benchmark] الـ Redis كلمة: 'product'
190 local.INFO: [Benchmark] الـ Redis كلمة: 'product'
191 local.INFO: [Benchmark] الـ Redis كلمة: 'product'
192 local.INFO: [Benchmark] الـ Redis كلمة: 'product'
193 local.INFO: [Benchmark] الـ Redis كلمة: 'product'

```

الطلب السابع : التحكم بالتزامن الموزع (Distributed Concurrency Control) :

في الأنظمة الموزعة عالية الأداء التي تعتمد على مجموعات خوادم متعددة (Multi-server cluster) ، تُصبح أقفال قواعد البيانات المحلية وحدها غير كافية لحماية النظام ومنع استهلاك الموارد غير المبرر. هدفنا في هذا الطلب هو تطبيق مفهوم الأقفال الموزعة (Distributed Locks) المبنية فوق خادم Redis، وإثبات كفاءتها في العمل كـ "حارس بوابة موحد" على مستوى الشبكة بالكامل (Cluster-Wide Application Logic).

- وتتجلى الأهمية في استخدام هذا المفهوم في :

حماية المنطق البرمجي : تأمين مسار العمليات الحساسة عبر الخوادم المتعددة قبل وصولها لطبقة البيانات.

- منع العمليات المكررة الخارجيّة : حظر المشاكل الكارثية مثل تكرار سحب الأموال (Double Payment) الناتج عن نقرات المستخدم المتزامنة.
- منع تداخل المهام الخلفية : ضمان عدم تشغيل الوظائف الموزعة المتزامنة
- (Distributed Background Jobs) من قبل أكثر من سيرفر في نفس اللحظة.

السيناريو الأول المتبع في الاختبار: محاكاة عملية تأكيد الطلب الافتراضية (Order Process Lifecycle)

تمر عملية الشراء وتثبيت الطلبية في مشروعنا بالخطوات الثلاث التالية:

- 1- **مرحلة الفحص والاستعلام (Read Phase)** : يستقبل النظام الطلب ويتجه فوراً لقراءة قاعدة البيانات للتحقق من شرط أساسي "هل هذا الطلب ما زال بحالة معلق (pending)؟" إذا تحقق الشرط، يُسمح للطلب بالمرور.
 - 2- **مرحلة المعالجة والاتصال (Processing Phase)** : يقوم التطبيق بالاتصال ببوابة دفع خارجية لسحب القيمة المالية. تتطلب هذه الخطوة زمناً عبر الشبكة، ونقوم بمحاكاتها برمجياً عبر أمر (sleep)
 - 3- **مرحلة التثبيت والحفظ (Write Phase)** : بعد ورود موافقة البوابة الخارجية يتم تحديث حالة الطلب في قاعدة البيانات إلى مدفوع (paid) لإغلاق المعاملة.
- نهدف في هذا الاختبار الى توضيح كيفية التعامل مع "الفجوة الزمنية" الواقعة بين الخطوة الأولى (قراءة البيانات) والخطوة الثالثة (تعديل البيانات وحفظها) واختبار كفاءة القفل الموزع في حال وجود أكثر من طلب .
- تم تصميم سيناريو تنافس متزامن بين طلبين كالتالي :

- حالة البيانات الابتدائية : نملك طلبية دفع (Order) في قاعدة البيانات تحمل معرفاً فريداً وحالتها معلقة status = 'pending'
- الهجوم المتزامن : يقوم مستخدمين اثنين بالنقر على زر "تأكيد الدفع" في نفس الملي ثانية.
- توزيع الطلبات بالتوازي : ينطلق الطلب الأول نحو الخادم الأول (Server A) ، بينما ينطلق الطلب الثاني المتزامن تماماً نحو الخادم الثاني (Server B) في نفس اللحظة الزمنية.

ملاحظة :

لأن سيرفر التطوير الافتراضي في لارافيل php artisan serve هو سيرفر أحادي الخيط (Single-threaded) ؛ أي أنه لا يستطيع معالجة أكثر من طلب واحد في نفس اللحظة على نفس المنفذ (Port) ؛ فإن بيئة PHP ستجبر الطلبات على التنفيذ بالتتالي (Sequentially) ، مما يخفي عيوب التزامن بالصدفة نتيجة لانتظار الطلبات خلف بعضها في طابور السيرفر.

لذلك واستناداً لبنية توزيع الأحمال، قمنا بتشغيل نسختين مستقلتين تماماً من خوادم التطبيق في آن واحد: الخادم الأول على المنفذ 8000 والخادم الثاني على المنفذ 8001 وذلك من أجل تطبيق محاكاة التوازي بشكل فعال

الاختبار والتحليل الهندسي :

1- إعدادات خوادم التطبيق وقاعدة البيانات :

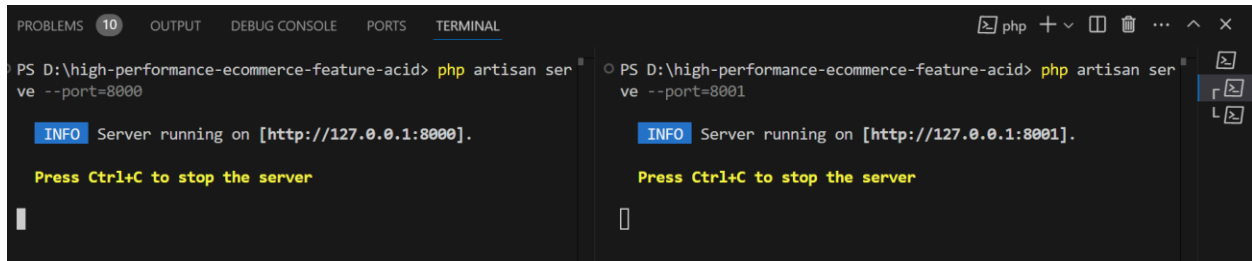
تم تشغيل نسختين مستقلتين تماماً من خادم Laravel عبر منافذ نظام تشغيل مختلفة لتعمل كـ Multi-Server Nodes :

- العقدة الأولى (Server Node A) : تعمل عبر المنفذ 8000 بالأمر

php artisan serve --port=8000

- العقدة الثانية (Server Node B) : تعمل عبر المنفذ 8001 بالأمر

php artisan serve --port=8001



تم ضبط الخادمين لقراءة الإعدادات المشتركة التالية من ملف التكوين .env.

- قاعدة البيانات الموحدة DB_DATABASE=high_performance_ecommerce :

لضمان قراءة وكتابة البيانات في مستودع موحد.

- محرك الأقفال المركزي (Redis) : تم ضبط REDIS_HOST=127.0.0.1 والاتصال بالمنفذ المشترك

REDIS_PORT=6379 ليكون Redis هو مدير الأقفال الموزع الشامل للشبكة بالكامل

هذا يضمن أن القفل الموزع (Distributed Lock) الذي يحجزه السيرفر الأول على منفذ 8000 سيراه السيرفر الثاني على منفذ 8001 فوراً في نفس الملي ثانية ويعرف أن المورد محجوز.

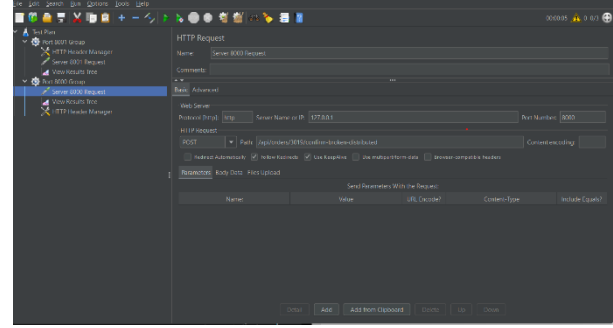
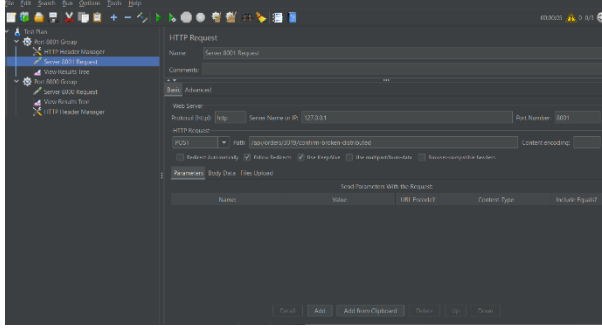
2- خطة الاختبار عبر أداة Apache JMeter :

تم بناء معمارية فحص متوازية حقيقية عبر العقد (Multi-Node Testing Architecture) عن طريق عزل الخوادم في مجموعات خيوط مستقلة كالتالي:

مجموعة الخيوط الأولى (Port 8000 Group) : تحتوي على مستخدم واحد يوجه طلب معالجة الدفع مباشرة للمنفذ 8000 .

مجموعة الخيوط الثانية (Port 8001 Group) : تحتوي على مستخدم متزامن تماماً يوجه طلبه مباشرة للمنفذ 8001 .

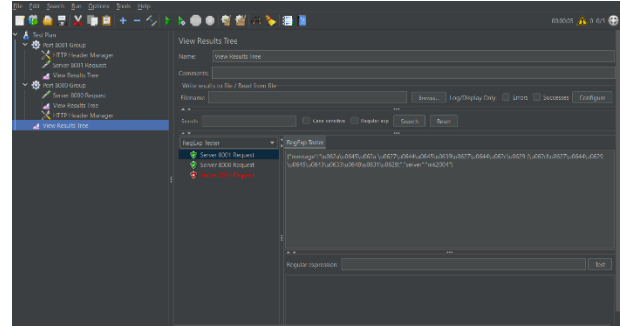
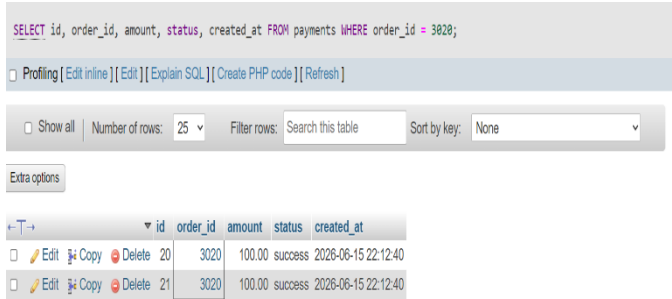
تم ضبط الـ 1 = Number of Threads لكل مجموعة، مع تصفير الـ 0 = Ramp-up period لضمان انطلاق الطلبين وصدمة السيرفرين في نفس الملي ثانية تماماً.



❖ الاختبار في حالة عدم استخدام الأقفال confirmOrderBrokenDistributed :

قمنا باستخدام sleep(3) لتعطيل السيرفر مؤقتاً ومحاكاة التأخير الزمني الشبكي الحاصل أثناء الاتصال ببوابة الدفع البنكية الخارجية. في هذه الثواني الثلاث، تظل حالة الطلب في الداتا بيز pending لأن السيرفر لم يحدثها بعد.

نقوم باختبار الراوتر /api/orders/{order_id}/confirm-broken-distributed بشكل متزامن على السيرفرين 8000 و 8001 بالتوازي لنرى :



تفسير النتائج : نلاحظ في أداة الاختبار أن الطلبين حصلوا على استجابة OK 200، وعند مراجعة سجلات قاعدة البيانات نجد أنه تم إنشاء سجلي دفع لنفس الطلب يحملان نفس الـ order_id ونفس التوقيت بالثانية وهنا تكمن مشكلة الـ Race Condition عبر السيرفرات.

❖ الاختبار في حالة استخدام الأقفال confirmOrderFixedDistributed :

التابع آلية حماية تعمل بالشكل التالي :

1- صياغة مفتاح القفل الفريد لهذا الطلب

```
$lockKey = 'lock:distributed:order:' . $order_id;
```

2- إنشاء القفل باستخدام أداة الأقفال (Cache::lock) بصلاحية 10 ثوانٍ (TTL) لمنع الـ Deadlock

```
$lock = Cache::lock($lockKey, 10);
```

3- بوابة الصد (if (!\$lock->get())) : التابع get() هو حارس البوابة الفعلي؛ يحاول الاستحواذ على المفتاح في Redis . إذا كان السيرفر الآخر قد سبقه بميكرو-ثانية واحدة، سيعيد التابع false . فوراً يتدخل شرط الـ if ليُطرد الطلب التنافسي المكرر ويرجع استجابة الرفض الفورية 429 Too Many Requests مع تسجيل رسالة تحذيرية في اللوغ Log::warning .

4- إذا نجح السيرفر في أخذ القفل، يدخل منطقة الأمان المعزولة لتنفيذ الدفع وتحديث الداتا بيز.

5- تحرير القفل (\$lock->release()) : تابع التحرير يُستدعى فوراً في نهاية خطة النجاح، أو داخل كتلة الـ catch في حال حدوث أي استثناء أو عطل طارئ، لفك القفل وإتاحة المورد.

نقوم باختبار الراوت /api/orders/{order_id}/confirm-fixed-distributed بشكل متزامن على السيرفرين 8000 و 8001 بالتوازي لنرى.

Showing rows 0 - 0 (1 total, Query took 0.0003 seconds.)

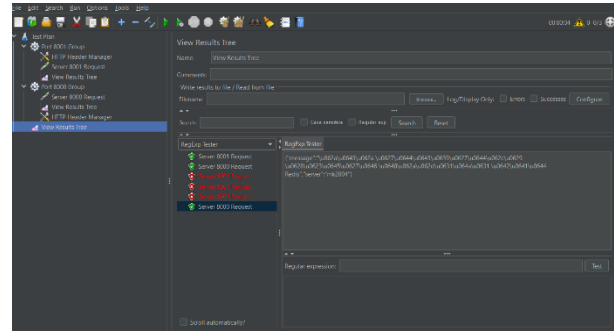
```
SELECT id, order_id, amount, status, created_at FROM payments WHERE order_id = 3021;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

	id	order_id	amount	status	created_at
	22	3021	100.00	success	2026-06-15 22:15:49



تفسير النتائج :

- الطلب الأول الصادر من السيرفر الفائز الخادم 8000 يعود بنجاح بلون أخضر OK 200 بعد معالجة دامت حوالي 3.9 ثواني.
- الطلب المتزامن الثاني الصادر من السيرفر الخاسر الخادم 8001 يتم صده فوراً ويتحول للون الأحمر ويكون الجواب 429 Too Many Requests
- فارق الزمن : الطلب المطرود بـ 429 استغرق زمناً لا يتجاوز 979ms! لم ينتظر في الطوابير ولم ينفذ أمر النوم، بل طُرد فوراً من طبقة الشبكة
- عند فحص قاعدة البيانات نجد سجل طلب واحد فقط تم تأكيده لا غير

السيناريو الثاني: الأقفال الموزعة لحماية المهام الخلفية المجدولة (Distributed Lock on Background Jobs)

في بيانات الخوادم المتعددة، يتم تشغيل المهام الدورية (مثل توليد تقارير المبيعات اليومية وجرد الأرباح) عبر وظائف خلفية مجدولة (Scheduled Background Jobs). إذا انطلقت هذه المهام في نفس الثانية من خوادم مختلفة دون نظام تنسيق موحد، ستحدث مشكلة تداخل المهام الحاسوبية، مما يؤدي إلى:

- إنهاك مزدوج لموارد المعالجة والذاكرة: تشغيل عمليات استعلام (Queries) ضخمة ومعقدة على قاعدة البيانات من عدة سيرفرات في نفس اللحظة.
- فساد وتكرار البيانات التقريرية: معالجة حزم البيانات وتقسيمها (Batch Chunks) مرتين، وتوليد سجلات مكررة في جداول التقارير.

يتم استخدام Redis Distributed Lock ليكون بمثابة منظم مركزي يضمن أن خادماً واحداً فقط في الشبكة بالكامل يملك الحق الحصري في إدارة وتوليد هذا التقرير اليومي، بينما تقوم الخوادم الأخرى بتخطي العملية فوراً بمجرد اكتشاف حزم القفل.

في مشروعنا عملية توليد التقرير اليومي تمر بالمراحل التالية :

1- **مرحلة الاستعلام والجمع (Query Chunks Phase) :** يقوم السيرفر ببدا الوظيفة، والاستعلام من قاعدة البيانات عن كافة الطلبات المدفوعة خلال اليوم .

2- **مرحلة التقسيم والمحاكاة (Batching & Sleep Phase) :** يتم تقسيم مصفوفة الطلبات الضخمة إلى حزم صغيرة لمعالجتها بالتوازي. ونقوم هنا بتوظيف أمر النوم (sleep(3 لمحاكاة الوقت المستغرق لتجهيز البيانات وفتح الفجوة الزمنية.

3- **مرحلة الإطلاق والتثبيت (Dispatch & Commit Phase) :** يتم إرسال الحزم معاً عبر نظام الـ Bus::batch لمعالجتها في الخلفية وتحديث جداول المبيعات.

تم تصميم سيناريو تنافس متزامن بين طلبين كالتالي :

حالة البيانات الابتدائية: قاعدة البيانات نظيفة، وجدول دفعات الوظائف الخلفية (job_batches) فارغ ولا يحتوي على أي دفعة مخصصة لتقرير اليوم.

الهجوم الموزع المتزامن: باستخدام أداة JMeter نقوم بإرسال طلبين متزامنين تماماً في نفس الملي ثانية. ينطلق الطلب الأول لضرب راوت الخادم الأول 8000 وينطلق الثاني لضرب راوت الخادم الثاني 8001 .

❖ الاختبار في حالة عدم استخدام الأقفال GenerateReportBrokenLock :

يدخل الخادمان معاً في نفس اللحظة:

- بمجرد إقلاع المهمة، يقوم كلا السيرفرين بتنفيذ سطر الـ sleep(3 معاً بالتوازي.
- **النتيجة:** يستيقظ السيرفران وكل منهما يظن أنه الوحيد المسؤول عن التقرير، فيقوم الخادم الأول بعمل Bus::batch(\$jobs)->dispatch() ويقوم الخادم الثاني بعمل

Bus::batch(\$jobs)->dispatch() مرة ثانية لنفس البيانات

نقوم باختبار الراوت /api/sales/report/generate-broken-lock بشكل متزامن على السيرفرين 8000 و 8001 بالتوازي لنرى.

```
PS D:\high-performance-e-commerce> Get-Content storage\logs\laravel.log -Tail 20 | Select-String "REPORT-BROKEN"
[2026-06-18 19:22:43] local.WARNING: [REPORT-BROKEN-LOCK] Server [mk2004] started report job for 2026-06-18 à€" NO distributed lock used. If another server
runs this concurrently, the report will be generated TWICE.
[2026-06-18 19:22:46] local.WARNING: [REPORT-BROKEN-LOCK] Server [mk2004] dispatched 0 chunk(s). Check job_batches table à€" if two rows exist for today,
the race condition occurred!
[2026-06-18 19:22:46] local.WARNING: [REPORT-BROKEN-LOCK] Server [mk2004] dispatched 0 chunk(s). Check job_batches table à€" if two rows exist for today,
the race condition occurred!
[2026-06-19 02:07:51] local.WARNING: [REPORT-BROKEN-LOCK] Server [mk2004] started report job for 2026-06-19 à€" NO distributed lock used. If another server
runs this concurrently, the report will be generated TWICE.
[2026-06-19 02:07:51] local.WARNING: [REPORT-BROKEN-LOCK] Server [mk2004] started report job for 2026-06-19 à€" NO distributed lock used. If another server
runs this concurrently, the report will be generated TWICE.
[2026-06-19 02:07:54] local.WARNING: [REPORT-BROKEN-LOCK] Server [mk2004] dispatched 0 chunk(s). Check job_batches table à€" if two rows exist for today,
the race condition occurred!
[2026-06-19 02:07:54] local.WARNING: [REPORT-BROKEN-LOCK] Server [mk2004] dispatched 0 chunk(s). Check job_batches table à€" if two rows exist for today,
the race condition occurred!
```

Showing rows 0 - 3 (4 total, Query took 0.0013 seconds.) [created_at: 1781834874... - 1781810566...]

SELECT name, total_jobs, created_at FROM job_batches WHERE name LIKE 'report-broken-lock%' ORDER BY created_at DESC;

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	name	total_jobs	created_at
☐ Edit ☐ Copy ☐ Delete	report-broken-lock-2026-06-19-mk2004	0	1781834874
☐ Edit ☐ Copy ☐ Delete	report-broken-lock-2026-06-19-mk2004	0	1781834874
☐ Edit ☐ Copy ☐ Delete	report-broken-lock-2026-06-18-mk2004	0	1781810566
☐ Edit ☐ Copy ☐ Delete	report-broken-lock-2026-06-18-mk2004	0	1781810566

تفسير النتائج :

1- في ملفات اللوغ: يطلع كلا الخادمين أسطر الـ Log::warning التي وضعناها بالكود والتي تؤكد بدء العمل المزدوج

2- في قاعدة البيانات نلاحظ وجود سطرين مكررين لنفس تقرير اليوم .

❖ الاختبار في حالة استخدام الأقفال GenerateReportFixedLock :

مراحل المعالجة :

1- يتم صياغة المفتاح بناءً على التاريخ الحالي لمنع تشغيل تقرير نفس اليوم مرتين باستخدام :

\$lockKey = 'lock:report-job:' . today()->toDateString();

2- يتم حجز القفل في Redis لمدة 10 دقائق (600 ثانية) بالأمر Cache::lock(\$lockKey, 600)

تم استخدام Time-To-Live لتحرير القفل تلقائياً في حال انهيار السيرفر فجأة أثناء المعالجة، منعاً لحدوث حالة Deadlock للمهمة .

3- السيرفر الذي يتم رفضه لا تتم معالجته وإنما فقط رفضه مع تسجيل Log::warning لإظهار النتائج

4- تحرير القفل بعد انتهاء الدفعة بالكامل يتم اطلاق القفل في جسم التابع (finally) المدمج في ال Batch لنضمن أن القفل لن يتحرر بمجرد إطلاق الوظيفة، بل سيظل محجوزاً في Redis طوال فترة قيام ال Workers بمعالجة الحزم في الخلفية، ويتم فكه فقط عند انتهاء آخر حزمة تماماً لضمان الأمان.

نقوم باختبار الراوت /api/sales/report/generate-fixed-lock بشكل متزامن على السيرفرين 8000 و 8001 بالتوازي لنرى.

Showing rows 0 - 2 (3 total, Query took 0.0007 seconds.) [created_at: 1781835503... - 1781810725...]

SELECT name, total_jobs, created_at FROM job_batches WHERE name LIKE 'report-fixed-lock%' ORDER BY created_at DESC;

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	name	total_jobs	created_at
☐ Edit ☐ Copy ☐ Delete	report-fixed-lock-2026-06-19	0	1781835503
☐ Edit ☐ Copy ☐ Delete	report-fixed-lock-2026-06-18	0	1781811620
☐ Edit ☐ Copy ☐ Delete	report-fixed-lock-2026-06-18	0	1781810725

```
PS D:\high-performance-e-commerce> Get-Content storage\logs\laravel.log -Tail 20 | Select-String "REPORT-FIXED"
[2026-06-18 19:25:22] local.INFO: [REPORT-FIXED-LOCK] Server [mk2004] acquired Redis lock and started report job for 2026-06-18
[2026-06-18 19:25:26] local.INFO: [REPORT-FIXED-LOCK] Server [mk2004] dispatched 0 chunk(s). Lock will be released after batch completes.
[2026-06-18 19:40:16] local.WARNING: [REPORT-FIXED-LOCK] Server [mk2004] tried to start report job for 2026-06-18 â€” REJECTED. Lock already held by another server. Skipping.
[2026-06-18 19:40:28] local.INFO: [REPORT-FIXED-LOCK] Server [mk2004] dispatched 0 chunk(s). Lock will be released after batch completes.
[2026-06-19 02:18:18] local.WARNING: [REPORT-FIXED-LOCK] Server [mk2004] tried to start report job for 2026-06-19 â€” REJECTED. Lock already held by another server. Skipping.
[2026-06-19 02:18:23] local.INFO: [REPORT-FIXED-LOCK] Server [mk2004] dispatched 0 chunk(s). Lock will be released after batch completes.
```

تفسير النتائج :

1- في ملفات اللوغ :ينجح أحد السيرفرات في أخذ القفل ويطبّع سطر النجاح

[REPORT-FIXED-LOCK] Server [Server-A] acquired Redis lock and started report job

وفي نفس الملي ثانية، يتلقّى السيرفر الثاني الرفض ويطبّع سطر التخطي

. [REPORT-FIXED-LOCK] Server [Server-B] tried to start report job — REJECTED. Skipping

2- في قاعدة البيانات : في جدول job_batches، يظهر سطر واحد مخصص لتقرير اليوم. تم تجاهل طلب السيرفر الثاني

الطلب الثامن : سلامة المعاملات / Transaction Integrity / ACID:

ان عملية شراء طلبية ما تخضع لثلاث مراحل:

1. اقتطاع كمية المنتج المراد شراؤه من المخزون
2. انشاء سجل يوثق عملية الشراء الخاصة بالطلب
3. تحديث حالة الطلب من "معلق" الى "مدفوع"

نلاحظ وجود عدة عمليات قراءة وكتابة ستتم على قاعدة البيانات, لذا فان حصل عطل في منتصف عملية شراء طلبية سيؤدي ذلك الى مشاكل في البيانات المسجلة ضمن قاعدة البيانات. لحل هذه المشكلة سنعمد تطبيق مبادئ ACID للتأكد من سلامة تنفيذ العملية كما يلي:

1. Atomicity:

سنعامل مع مختلف عمليات القراءة والكتابة كنواة واحدة – أي ان عملية القراءة والكتابة ستتم مرة واحدة وليس عدة مرات موزعة على ازمة مختلفة. كما سيتم حماية قاعدة البيانات من التعديلات الجزئية (بحال حدث عطل بعد عملية كتابة جزئية فسيتم التراجع عن كل عمليات الكتابة التي جرت)

2. Consistency:

كل البيانات ضمن قاعدة البيانات ستكون متسقة – لن يتواجد سجل بحاجة لبيانات مفقودة او غير منطقية بالمقارنة مع السجل (مثال: وجود سجل عملية دفع لطلب ولكن حالة الطلب مازالت "معلق")

3. Isolation:

سيتم عزل كل عملية عن الأخرى – لن يتم السماح بعمليات قراءة وكتابة متداخلة (ذلك سيتم عبر استخدام أقفال متشائمة تمنع القراءة والكتابة حتى تنهي العملية كل عملها على قاعدة البيانات).

4. Durability:

بعد ان تجري العملية ونكتب على قاعدة البيانات, يجب حتما ان تتم كتابة التعديلات على قاعدة البيانات بغض النظر عن حصول عطل في قاعدة البيانات او أي مشكلة أخرى...

تم تنفيذ المعايير السابقة عبر استخدام التالي

- `DB::rollBack()`: الذرية والاتساق - تراجع عن جميع الخطوات إذا فشلت أي منها
- `DB::commit()`: المتانة - البيانات مكتوبة بشكل دائم على القرص (تتولى خاصية InnoDB WAL في MySQL تلقائيًا عبر `DB::commit()`، ولا حاجة إلى أي كود برمجي للتطبيق).
- `lockForUpdate()`: العزل – قفل الصف لنهاية المعاملة

الاختبارات:

تم اختبار ثلاثة سيناريوهات. يُظهر كل سيناريو حالة قاعدة البيانات للمدفوعات والطلبات والمنتجات بعد الطلب.

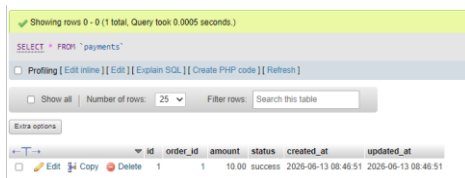
الحالة الابتدائية: المنتج يملك 100 قطعة بالمخزن – يوجد طلب معلق لشراء قطعة واحدة من ذاك المنتج - لا يوجد عملية شراء مسبقة

1. السيناريو 1 – تابع غير محسن مع عطل مُحاكى:

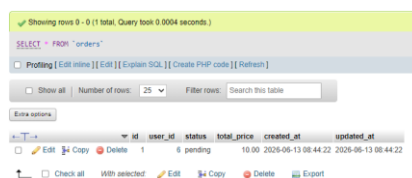
التابع ينفذ العمليات الثلاثة بدون مراعاة للمعايير الموضوعة – كما يحوي على trigger يقوم بإلقاء خطأ قبل الخطوة 3 (تحديث حالة الطلب) عبر باراميتز يمرر يدويا لمحاكاة حالة حدوث مشكلة او عطل

POST /api/orders/1/confirm-broken-acid?simulate_fail=1

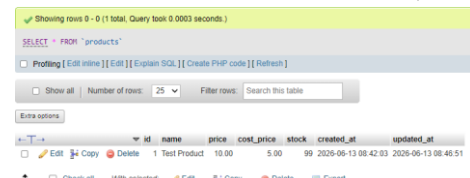
بعد اجراء الاختبار حصلنا على حالة غير متسقة: تم تحصيل المبلغ وكتابة السجل الخاص به، وتم خصم المخزون، ولكن لم يتم تغيير حالة الطلب مطلقاً.



id	order_id	amount	status	created_at	updated_at
1	1	10.00	success	2020-06-13 08:45:51	2020-06-13 08:45:51



id	user_id	status	total_price	created_at	updated_at
1	6	pending	10.00	2020-06-13 08:44:22	2020-06-13 08:44:22



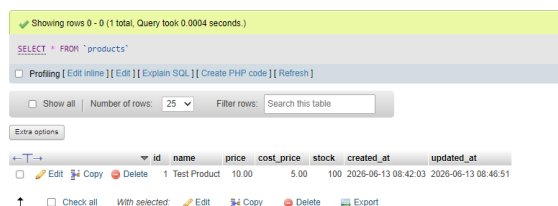
id	name	price	cost_price	stock	created_at	updated_at
1	Test Product	10.00	5.00	99	2020-06-13 08:42:03	2020-06-13 08:45:51

1. السيناريو 2 – تابع محسن مع عطل مُحاكى:

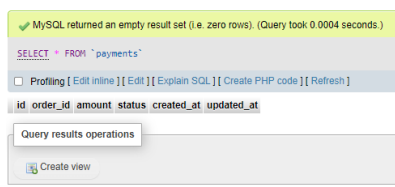
التابع ينفذ العمليات الثلاثة مع مراعاة للمعايير الموضوعة – كما يحوي على trigger يقوم بإلقاء خطأ قبل الخطوة 3 (تحديث حالة الطلب) عبر باراميتز يمرر يدويا لمحاكاة حالة حدوث مشكلة او عطل

POST /api/orders/1/confirm-fixed-acid?simulate_fail=1

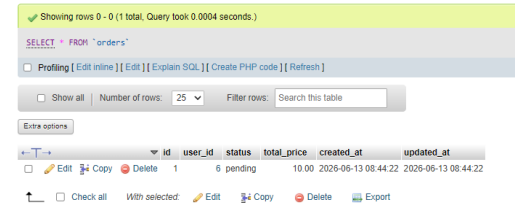
بعد اجراء الاختبار حصلنا على حالة متسقة: التراجع الكامل - قاعدة البيانات مطابقة لما كانت عليه قبل الطلب. لا يوجد أي أثر لعمليات الكتابة الجزئية.



id	name	price	cost_price	stock	created_at	updated_at
1	Test Product	10.00	5.00	100	2020-06-13 08:42:03	2020-06-13 08:45:51



id	order_id	amount	status	created_at	updated_at
----	----------	--------	--------	------------	------------



id	user_id	status	total_price	created_at	updated_at
1	6	pending	10.00	2020-06-13 08:44:22	2020-06-13 08:44:22

1. السيناريو 3 – تابع محسن بدون عطل مُحاكى:

نفس التابع السابق لكن من دون محاكاة لعطل اثناء العمل

POST /api/orders/1/confirm-fixed-acid

بعد اجراء الاختبار حصلنا على حالة متسقة: تم تحديث الجداول الثلاثة جميعها بشكل متزامن بالنهاية.

Showing rows 0 - 0 (1 total, Query took 0.0004 seconds.)									
SELECT * FROM `products`									
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]									
Show all Number of rows: 25 Filter rows: Search this table									
Extra options									
+ T +									
id name price cost_price stock created_at updated_at									
Edit Copy Delete 1 Test Product 10.00 5.00 99 2026-06-13 08:42:03 2026-06-13 08:51:53									
Check all With selected Edit Copy Delete Export									

Showing rows 0 - 0 (1 total, Query took 0.0005 seconds.)									
SELECT * FROM `payments`									
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]									
Show all Number of rows: 25 Filter rows: Search this table									
Extra options									
+ T +									
id order_id amount status created_at updated_at									
Edit Copy Delete 3 1 10.00 success 2026-06-13 08:51:53 2026-06-13 08:51:53									
Check all With selected Edit Copy Delete Export									

Showing rows 0 - 0 (1 total, Query took 0.0005 seconds.)									
SELECT * FROM `orders`									
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]									
Show all Number of rows: 25 Filter rows: Search this table									
Extra options									
+ T +									
id user_id status total_price created_at updated_at									
Edit Copy Delete 1 6 paid 10.00 2026-06-13 08:44:22 2026-06-13 08:51:53									
Check all With selected Edit Copy Delete Export									

الطلب التاسع: اختبار الاستقرار تحت الضغط العالي (Stress Testing Report)

مقدمة :

يهدف هذا الاختبار إلى دراسة وتحليل سلوك محرك المعالجة عالي الأداء للنظام تحت ظروف الضغط الشديد والوصول المتزامن الكثيف (High Concurrency) تم تنظيم الاختبار لإثبات قدرة النظام على تخديم 100 مستخدم متزامن (Concurrent Users) في اللحظة نفسها دون حدوث انهيار كلي في الخدمة أو تلف في تناسق البيانات

الأهداف الهندسية المحددة للاختبار:

1. قياس استجابة النظام عند وصول عدد الخيوط الفعالة (Active Threads) إلى الحد الأقصى المستهدف N=100
2. كشف نقاط الاختناق (Bottlenecks) الناتجة عن تضارب الخيوط على الموارد المشتركة، وتحديداً عند تعديل كميات المخزون الحساسة.
3. التحقق من فعالية طبقة التخزين المؤقت (Caching Layer) في تخفيف العبء عن قاعدة البيانات أثناء ذروة الضغط.
4. إثبات سلامة المعاملات البرمجية وتطبيق معايير (ACID Integrity) عبر مراقبة سلوك النظام عند تراجع العمليات المتصادمة (Rollback)

مواصفات بيئة الاختبار :

تم إعداد أداة الفحص المستخدمة لمحاكاة الحمل الرقمي وفق المعايير البرمجية التالية :

- أداة الاختبار المعتمدة : Apache JMeter
- عدد الـ **thread المستخدمة** : تم تحديد 100 مستخدم متزامن يتم حقنهم في النظام بشكل تدريجي متسارع لمحاكاة أوقات الذروة
- المسارات المستهدفة بالضغط :
تم التركيز على مسارات أساسية مثل :
 - **POST /api/login** : واجهة التحقق من الهوية (Authentication) ، وهي واجهة مكثفة حاسوبياً بسبب عمليات تشفير وفك تشفير الرموز (Tokens) .

- GET /api/products-search : البحث الديناميكي عن المنتجات، وتستهدف قياس سرعة استعلامات قاعدة البيانات تحت الضغط.
- POST /api/cart/{product_id}/add-fixed : api المخصص لإضافة المنتجات للسلة وحجز المخزون، وهي المحمية بآليات العزل ومنع التضارب.
- GET /api/cache/top-products: واجهة استعراض المنتجات الأكثر طلباً، وتعتمد بالكامل على طبقة التخزين المؤقت (Cache) لفك الارتباط عن قاعدة البيانات.
- POST /api/orders/confirm-fixed : تأكيد الطلب (Confirm Order Fixed ACID) ، وهو api حرج جداً تقوم بعمليات مركبة خلف بعض (تحديث المخزون، إنشاء الفاتورة) وبتحملهم قواعد ال-ACID من أجل أن تضمن نجاح العمليات كلها أو فشلها كلها بدون ضياع بيانات.

سيناريوهات الاختبار المنفذة :

لفهم سلوك النظام وقاعدة البيانات بشكل دقيق تحت الضغط العالي، تم تصميم وتنفيذ سيناريوهين مختلفين :
أولاً: سيناريو دورة حياة الطلب الكاملة :

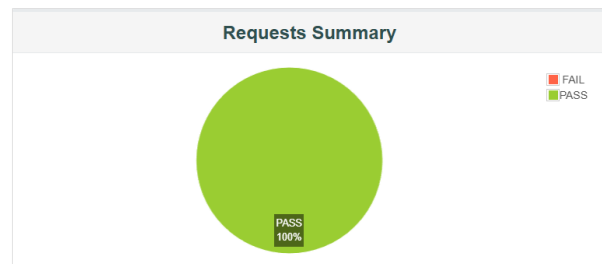
100 يوزر في دورة واحدة لا تتجاوز الـ 5 ثانية و تضمن تأكيد طلب الشراء :

- **وصف السيناريو :** يقوم 100 مستخدم متزامن بالدخول إلى النظام في نفس اللحظة، وينفذ كل مستخدم الدورة البرمجية الكاملة لمرة واحدة فقط وبشكل متتابع.
- **خطوات السير البرمجي :** تبدأ العملية بتسجيل الدخول، ثم البحث عن منتج، تليها إضافة المنتج للسلة، وتنتهي بتأكيد الطلب وحجز المخزون الفعلي.
- **الهدف من السيناريو :** التأكد من سلامة التتابع البرمجي (Sequential Integrity) عند حدوث حمل مفاجئ، والتحقق من عدم تداخل جلسات المستخدمين (User Sessions) أثناء إتمام العمليات ووصولها إلى قاعدة البيانات.

في هذا الاختبار لم نجد أخطاء :

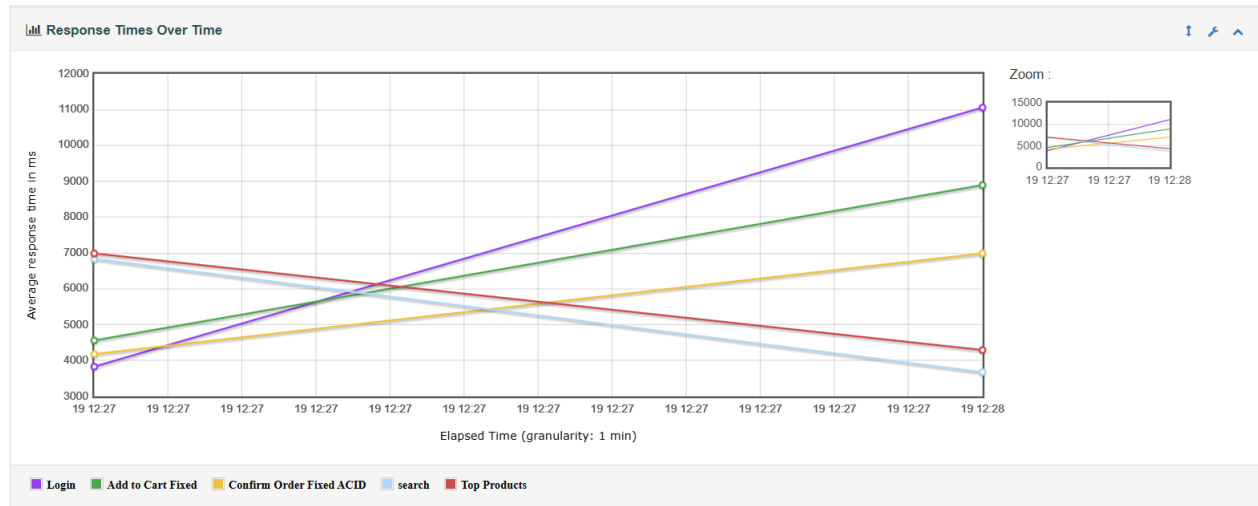
Test and Report information	
Source file	"results.jtl"
Start Time	"6/19/26, 12:27 PM"
End Time	"6/19/26, 12:28 PM"
Filter for display	""

APDEX (Application Performance Index)			
Apdex	T (Toleration threshold)	F (Frustration threshold)	Label
0.055	500 ms	1 sec 500 ms	Total
0.025	500 ms	1 sec 500 ms	Login
0.033	500 ms	1 sec 500 ms	Add to Cart Fixed
0.075	500 ms	1 sec 500 ms	Confirm Order Fixed ACID
0.083	500 ms	1 sec 500 ms	Top Products
0.175	500 ms	1 sec 500 ms	search



Statistics

Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent
Total	200	0	0.00%	6862.36	448	16495	6647.50	13277.00	14546.90	15564.46	6.53	4.97	2.66
Add to Cart Fixed	30	0	0.00%	8173.50	1408	15412	8208.50	15060.60	15231.60	15412.00	1.06	0.40	0.60
Confirm Order Fixed ACID	20	0	0.00%	6567.45	992	14647	6349.50	13513.60	14599.75	14647.00	0.69	0.21	0.36
Login	100	0	0.00%	7878.46	1116	16495	7843.00	13294.20	14419.75	16485.71	3.83	2.37	1.12
search	20	0	0.00%	3831.85	448	14553	2490.00	10744.40	14370.50	14553.00	0.75	1.61	0.38
Top Products	30	0	0.00%	4381.13	470	13848	2661.00	11764.10	13446.50	13848.00	1.16	1.16	0.57



أثبت السيناريو الأول بشكل قاطع أن النظام قادر على تخديم 100 مستخدم متزامن يُنفذون دورة الطلب الكاملة في آنٍ واحد دون أي خطأ أو فقدان بيانات. نسبة النجاح 100% عبر جميع الواجهات بما فيها Confirm Order Fixed ACID التي تُعدّ الأخرج هندسياً، تؤكد سلامة آليات الـ ACID والـ Locking المُطبقة في النظام. هذا السيناريو يُثبت أن النظام يعمل بشكل متكامل، وأن جلسات المستخدمين لا تتداخل.

تم أرفاق المجلد كامل بالإضافة إلى آلية انشاء اختبار (لكن لكي ينجح يرجى تعديل الـ path في الـ jmeter و الأمر الخاص بإنشاء ملف الـ csv)

ثانياً: سيناريو الإجهاد المستمر وعزل المخزون:

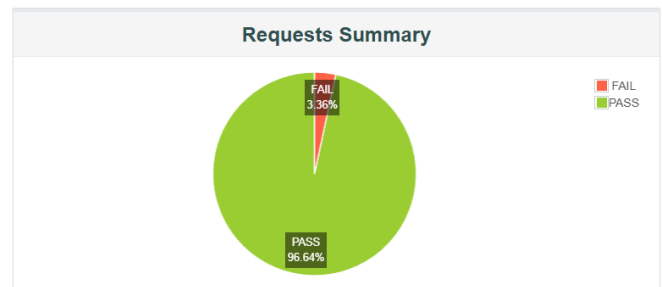
100 يوزر مع 50 دورة متتالية لإنشاء ضغط مستدام على واجهة السلة والمخزون:

- **وصف السيناريو :** يقوم 100 مستخدم متزامن بالدخول إلى النظام في نفس الوقت، وتكرار العمليات لـ 50 دورة (Loops) متتالية دون أي توقف، مما يولد آلاف الطلبات المتلاحقة والمستمرة على الخادم في فترة زمنية ممتدة.

- **خطوات السير البرمجي :** تتضمن العمليات تكرار تسجيل الدخول، ثم البحث الديناميكي عن المنتجات، يليه الضغط المكثف والمتتابع على إضافة المنتج للسلة عبر الـ API المصلح (add-fixed) ، مع استبعاد خطوة تأكيد الطلب النهائية لعزل واجهة السلة ودراسة سلوكها بشكل منفصل.
- **الهدف من السيناريو :** وضع واجهة التعديل على المخزون تحت حمل متواصل ومستدام لكشف مدى تزامم الخيوط (Thread Contention) على قاعدة البيانات والموارد المشتركة، واختبار جدار الحماية البرمجي ضد ظاهرة التنافس (Race Conditions) عند محاولة تعديل نفس المنتج في ذات الوقت.

Test and Report information	
Source file	"results.jtl"
Start Time	"6/19/26, 9:18 PM"
End Time	"6/19/26, 9:38 PM"
Filter for display	""

APDEX (Application Performance Index)			
Apdex	T (Toleration threshold)	F (Frustration threshold)	Label
0.002	500 ms	1 sec 500 ms	Total
0.001	500 ms	1 sec 500 ms	Add to Cart fixed
0.001	500 ms	1 sec 500 ms	Login
0.002	500 ms	1 sec 500 ms	search
0.004	500 ms	1 sec 500 ms	Top Products



Statistics													
Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label ^	#Samples ^	FAIL ^	Error % ^	Average ^	Min ^	Max ^	Median ^	90th pct ^	95th pct ^	99th pct ^	Transactions/s ^	Received ^	Sent ^
Total	9000	302	3.36%	12771.52	2	26733	13014.50	14525.00	15006.95	22355.97	7.49	7.71	4.16
Add to Cart fixed	1500	301	20.07%	12616.82	3	26733	12905.00	14346.60	14855.35	22668.73	1.25	2.09	0.70
Login	5000	1	0.02%	12983.88	2	26661	13214.50	14696.70	15150.00	22844.94	4.16	2.59	2.43
search	1000	0	0.00%	12543.99	546	26432	12771.00	14242.10	14674.95	22472.51	0.83	1.78	0.42
Top Products	1500	0	0.00%	12370.06	461	26557	12672.50	14104.80	14593.45	21982.83	1.25	1.25	0.62

في هذا الاختبار قمنا بتعطيل تأكيد الطلب لأننا نحتاج معرف طلب من أجل تأكيده و لا نستطيع تخزين معرفات مسبقة لكامل الطلبات وإن ابقينا التأكيد سيتم تأكيد الطلبات في الدورة الأولى و الدورة الثانية سيظهر خطأ لأن الطلبات تم تأكيدها.

Errors

Type of error	Number of errors	% in errors	% in all samples
429/Too Many Requests	299	99.01%	3.32%
Non HTTP response code: java.net.SocketException/Non HTTP response message: An established connection was aborted by the software in your host machine	1	0.33%	0.01%
Non HTTP response code: org.apache.http.NoHttpResponseException/Non HTTP response message: 127.0.0.1:8080 failed to respond	1	0.33%	0.01%
500/Internal Server Error	1	0.33%	0.01%

شرح الأخطاء :

96.64% من الأخطاء كانت Too Many Requests

السيرفر عنده Rate Limiter (حد أقصى للطلبات في الثانية)، وعندما تجاوز 100 مستخدم $\times$ 50 دورة هذا الحد، السيرفر رفض الطلبات تلقائياً ليس crash ، بل دفاع مقصود من السيرفر.

Connection Aborted

An established connection was aborted by the software in your host machine
 من جانب الـ OS أو الـ client قبل اكتمال الرد. يحدث عادةً عند ضغط لحظي شديد على الجهاز نفسه أثناء الاختبار. حالة نادرة جداً حدثت مرة واحدة

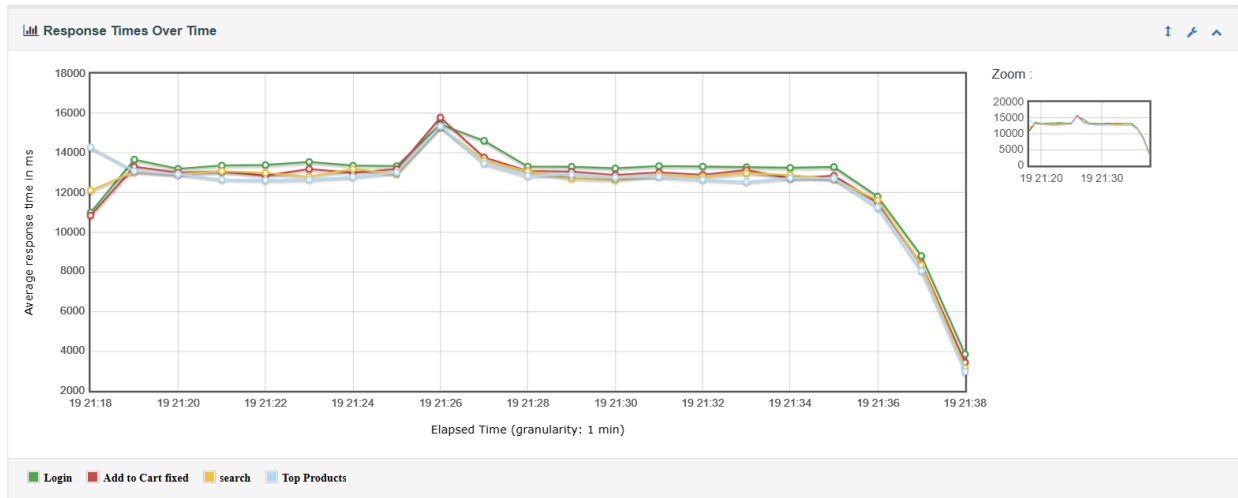
Failed to Respond

السيرفر قبل الطلب لكن لم يرد أبداً حتى انتهى الـ timeout حالة نادرة من الإجهاد الشديد.

Internal Server Error

خطأ داخلي في السيرفر نفسه على الأرجح Deadlock لحالة استثنائية لم يتم اكتشافها .

و الآن مخطط الاستجابة :



آلية إنشاء :

تم إنشاء تعليمة php artisan jmeter:generate ستولد ملف csv حاوي على user , product , order جديدة

(تحديدا user_id,product_id,order_id,email,password) نغذيهِ لملف الjmx

ملاحظة : يجب أن يعدل المسار الخاص بحفظ الملف يوجد داخل
App\Console\Commands\GenerateJmeterData.php

بالإضافة الى الpath داخل الاختبار

الاستنتاج النهائي:

أثبتت نتائج الاختبارين مجتمعين أن النظام حقق الهدف الهندسي المحدد وهو تخديم 100 مستخدم متزامن دون انهيار كلي أو فقدان بيانات بشكل كلي. السيناريو الأول أثبت سلامة دورة الحياة الكاملة بنسبة نجاح 100%، بما فيها العمليات الحرجة كـ Confirm Order Fixed ACID. أما السيناريو الثاني فكشف أن نقطة الاختناق الوحيدة هي Add to Cart api تحت الإجهاد المستمر (50 دورة)، وأن 99% من أخطائها ناتجة عن الـ Rate Limiter. لم يُسجل أي تلف في البيانات طوال فترة الاختبار.

الطلب العاشر : الاختناقات (Bottlenecks)

أولاً: الهدف

يرمي هذا المحور إلى قياس أوقات استجابة واجهات برمجة التطبيقات (API) عبر جميع وحدات النظام، وتحديد نقطة الاختناق الأبطأ (Bottleneck)، وتقديم مقارنة تحليلية رقمية دقيقة بين حالتي النظام قبل التحديثات وبعدها؛ بهدف إثبات فاعلية التحسينات البرمجية التي أجراها فريق التطوير.

ثانياً: منهجية العمل

اعتمدت المنهجية على تأسيس نظام تسجيل ومراقبة يستند إلى برمجيات وسيطة (Middleware)، لالتقاط زمن استغراق كل طلب (Request) وتوجيه سجلات الأداء إلى ملفات مخصصة لكل وحدة برمجية على حدة. كما وُفِّرت أوامر برمجية تحليلية تتيح مقارنة مستويات الأداء بدقة قبل عمليات التحسين وبعدها.

ثالثاً: تفاصيل التنفيذ

1- البنية التحتية للسجلات (Logging Infrastructure)

شملت هذه المرحلة تخصيص قنوات تسجيل مستقلة في ملف الإعدادات لتغطية كافة وحدات النظام: الهوية والصلاحيات، المنتجات، الطلبات، المخزون، والمدفوعات، إضافةً إلى قناة موحدة لقياس الأداء الإجمالي. كما جرى توجيه بيانات كل قناة برمجية لتُكتب في ملف مستقل داخل مسار السجلات، لضمان عزل البيانات وسهولة تتبعها.

2- برمجية مراقبة الأداء الوسيطة (Performance Monitoring Middleware)

طُوِّرت برمجية وسيطة مُدمجة كلياً في النظام تمر من خلالها جميع الطلبات، وتعمل تلقائياً عند كل طلب وفق الخطوات الآتية:

1. التقاط الطابع الزمني بدقة الميكروثانية فور وصول الطلب.
2. تمرير الطلب إلى وحدة التحكم (Controller) لمعالجته.
3. التقاط الطابع الزمني عند اكتمال المعالجة، وحساب الفارق الزمني بالميلي ثانية.
4. التعرف الذكي على الوحدة المعنية بناءً على رابط الطلب (مثل توجيه طلبات تسجيل الدخول إلى وحدة الصلاحيات، وطلبات الشراء إلى وحدة الطلبات).
5. تدوين سطر سجل موحد ومعياري يتضمن: التوقيت، اسم الوحدة، الرابط المستهدف، نوع الطلب، الزمن المستغرق، رمز حالة الاستجابة، والمعرف الفريد للطلب.

3- تحليل السجلات (Log Analysis)

بُنِيَ أمر برمجي مخصص عبر سطر الأوامر لتحليل قياسات الأداء، يؤدي الوظائف التالية:

- قراءة ملفات السجلات المخزنة في المسارات المخصصة لمرحلتَي "ما قبل التحسين" و "ما بعد التحسين".
- تجميع البيانات وتصنيفها بحسب الرابط المستهدف (Endpoint).

- حساب معدلات زمن الاستجابة، ومؤشر النسبة المئوية الـ 95 (P95) ، والحد الأقصى لزمن الاستجابة، ونسبة الخطأ لكل واجهة برمجية.
- عرض جداول مقارنة تُبين نسب التحسن بوضوح ودقة.
- تحديد نقطة الاختناق الأساسية في النظام استناداً إلى أعلى متوسط زمن استجابة أو أعلى قيمة لمؤشر P95.

4-نظام حفظ اللقطات(Snapshot System)

طُوّر أمر برمجي مخصص لالتقاط الحالة الزمنية للأداء وأرشفتها سواء قبل التحديث أو بعده، إذ يقوم بنسخ السجلات الحالية لوحداث النظام ونقلها إلى مجلدات الأرشفة المخصصة، مع تفريغ ملفات السجلات النشطة لضمان نظافة البيانات وجاهزيتها للاختبارات التالية دون أي تدخل.

رابعاً: النتائج

كشفت التحليلات الصادرة عن النظام أن نقطة الاختناق الرئيسية كانت تكمن في واجهة تأكيد الطلبات POST `/api/orders/{order_id}/confirm-broken`، وقد تجلّى ذلك في الارتفاع الملحوظ في متوسط زمن الاستجابة ومؤشر P95 تحت الضغط التشغيلي العالي والطلبات المتزامنة.

وعقب تطبيق فريق التطوير لحزمة من التحسينات الهيكلية، شملت تفعيل تقنيات قفل قواعد البيانات، والتخزين المؤقت عبر Redis، ومعالجة الطوابير غير المتزامنة، وفرض القيود الموزعة؛ سجّلت الواجهة البرمجية ذاتها انخفاضاً حاداً وملحوظاً في أوقات الاستجابة، مما أثبت بالدليل الرقمي القاطع نجاح الحلول التقنية المعتمدة وفعاليتها.